

Mục lục

Mục lục	1
Danh sách hình vẽ	3
Danh sách bảng	4
1 TỔNG QUAN	5
1.1 Giới thiệu đề tài	5
1.1.1 Bối cảnh	5
1.1.2 Mục tiêu	6
1.2 Cơ sở lý thuyết	7
1.2.1 Nền tảng Android và ngôn ngữ Java	7
1.2.2 Kiến trúc Client–Server và REST API	7
1.2.3 Kiến trúc phân lớp và mẫu Repository	7
1.2.4 Vòng đời (Lifecycle) trong Android	8
1.2.5 Giao tiếp HTTP với Retrofit và OkHttp	8
1.2.6 Phát video trực tuyến với ExoPlayer (Media3)	9
1.2.7 Tải và cache ảnh với Glide	9
1.2.8 Điều hướng với ViewPager2 và BottomNavigationView	9
1.2.9 Hệ thống thiết kế Material Design	10
1.2.10 Phát triển backend với Spring Boot	10
1.2.11 Cơ sở dữ liệu PostgreSQL và quản lý schema với Flyway	10
1.2.12 Xác thực không trạng thái với JSON Web Token	11
2 THIẾT KẾ HỆ THỐNG	12
2.1 Phân tích yêu cầu hệ thống	12
2.1.1 Yêu cầu chức năng (Functional Requirements)	12
2.1.2 Yêu cầu phi chức năng (Non-Functional Requirements)	23
2.2 Thiết kế lược đồ dữ liệu	25
2.2.1 Nhóm Tài khoản và Gói cước	25
2.2.2 Nhóm Nội dung phim	25
2.2.3 Nhóm Tương tác người dùng	26
2.2.4 Quản lý phiên bản schema với Flyway	26

2.3	Sơ đồ Tuần tự Nghiệp vụ (Sequence Diagrams)	27
2.4	Kiến trúc tổng thể	35
2.4.1	Sơ đồ Kiến trúc Tổng thể	35
2.4.2	Cơ chế giao tiếp Client–Server	36
2.4.3	Áp dụng Mẫu thiết kế (Design Patterns)	38
2.5	Thiết kế cấu hình các node và mạng	40
2.5.1	Cấu trúc Phân vùng Mạng	41
2.5.2	Cấu hình Môi trường Phát triển	42
2.5.3	Cấu hình cho Thiết bị Android Thật	42
3	TRIỂN KHAI HỆ THỐNG	43
3.1	Môi trường và công nghệ triển khai	43
3.1.1	Ngôn ngữ và Framework	43
3.1.2	Cấu hình ứng dụng và cơ sở dữ liệu	43
3.1.3	Môi trường phát triển	44
3.2	Kết quả triển khai các chức năng chính	44
3.2.1	Xác thực và Quản lý tài khoản (UC-01 – UC-05)	44
3.2.2	Khám phá nội dung (UC-06 – UC-10)	45
3.2.3	Quản trị phim và danh mục (UC-11 – UC-15)	46
3.3	Đánh giá kết quả thực nghiệm	47
3.3.1	Mức độ hoàn thành Use Case	47
3.3.2	Đánh giá hiệu năng	47
3.3.3	Đánh giá tính nhất quán dữ liệu	48
3.3.4	Đánh giá bảo mật cơ bản	49
4	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	50
4.1	Những kết quả đã đạt được	50
4.2	Hạn chế và hướng phát triển	51
4.2.1	Hạn chế hiện tại	51
4.2.2	Hướng phát triển ngắn hạn (3–6 tháng)	52
4.2.3	Hướng phát triển dài hạn (>6 tháng)	53
	TÀI LIỆU THAM KHẢO	54
A	BẢNG PHÂN CÔNG CÔNG VIỆC	56

Danh sách hình vẽ

2.1	Sơ đồ Use Case Tổng quát của ứng dụng CineFlow	12
2.2	Sơ đồ Thực thể Liên kết (ERD) của hệ thống CineFlow	26
2.3	Sơ đồ Tuần tự: UC-01 Đăng ký tài khoản	27
2.4	Sơ đồ Tuần tự: UC-02 Đăng nhập hệ thống	28
2.5	Sơ đồ Tuần tự: UC-03 Quên mật khẩu	28
2.6	Sơ đồ Tuần tự: UC-04 Đặt lại mật khẩu	29
2.7	Sơ đồ Tuần tự: UC-05 Đăng xuất	29
2.8	Sơ đồ Tuần tự: UC-06 Xem trang chủ	30
2.9	Sơ đồ Tuần tự: UC-07 Xem chi tiết phim	30
2.10	Sơ đồ Tuần tự: UC-08 Phát video	31
2.11	Sơ đồ Tuần tự: UC-09 Xem video ngắn	31
2.12	Sơ đồ Tuần tự: UC-10 Xem lịch Premier League	32
2.13	Sơ đồ Tuần tự: UC-11 Xem Dashboard quản trị	32
2.14	Sơ đồ Tuần tự: UC-12 Tạo phim mới	33
2.15	Sơ đồ Tuần tự: UC-13 Cập nhật phim	33
2.16	Sơ đồ Tuần tự: UC-14 Xoá phim	34
2.17	Sơ đồ Tuần tự: UC-15 Quản lý danh mục thể loại	34
2.18	Sơ đồ Kiến trúc Tổng thể của hệ thống CineFlow	36
2.19	Sơ đồ Tuần tự minh hoạ luồng đăng nhập và đính kèm JWT	37
2.20	Sơ đồ Tuần tự minh hoạ luồng phát video streaming	38
2.21	Sơ đồ Triển khai (Deployment Diagram) các Node và Phân vùng mạng	41
3.1	Giao diện Đăng ký và Đăng nhập tài khoản	45
3.2	Giao diện Quên mật khẩu và Đặt lại mật khẩu	45
3.3	Giao diện Trang chủ (Home) với các khu vực nội dung	46
3.4	Giao diện Chi tiết phim và Phát video	46
3.5	Giao diện Shorts (video ngắn dạng cuộn dọc)	46
3.6	Giao diện Lịch thi đấu Premier League	46
3.7	Giao diện Dashboard quản trị (Admin)	47
3.8	Giao diện Danh sách phim (Admin) và Form tạo/sửa phim	47

Danh sách bảng

2.1	Yêu cầu phi chức năng của hệ thống CineFlow	24
3.1	Ngôn ngữ, Framework và thư viện chính	43
3.2	Môi trường phát triển và công cụ	44
3.3	Kết quả đo hiệu năng các API chính	48
4.1	Bảng đối chiếu mục tiêu và kết quả thực hiện	50

CHƯƠNG 1

TỔNG QUAN

1.1 Giới thiệu đề tài

1.1.1 Bối cảnh

Trong những năm gần đây, hành vi tiêu thụ nội dung giải trí của người dùng tại Việt Nam và trên toàn cầu đã dịch chuyển mạnh mẽ từ truyền hình truyền thống sang các nền tảng phát trực tuyến (streaming) trên thiết bị di động nhờ sự phổ biến của các dịch vụ như Netflix, Disney+, FPT Play hay Galaxy Play. Mô hình này không chỉ đem lại trải nghiệm xem phim linh hoạt theo nhu cầu cá nhân mà còn mở ra một thị trường nội dung số khổng lồ với hàng tỷ giờ xem mỗi năm.

Tuy nhiên, đằng sau giao diện xem phim đơn giản là những bài toán kỹ thuật phức tạp mà các hệ thống này phải giải quyết:

- **Phát video trực tuyến hiệu năng cao:** Trình phát phải xử lý được các định dạng streaming hiện đại (HLS, DASH), thích nghi chất lượng theo băng thông và quản lý chặt chẽ vòng đời của các thành phần giao diện trên di động.
- **Tổ chức nội dung phong phú:** Trang chủ một ứng dụng streaming thường có nhiều khu vực nội dung khác nhau (banner, phim mới, series nổi bật, sự kiện thể thao), yêu cầu một mô hình dữ liệu và cơ chế tải bất đồng bộ đủ linh hoạt.
- **Xác thực và phân quyền:** Hệ thống cần phân biệt rõ ràng giữa người xem thông thường và quản trị viên — đây là bài toán xác thực không trạng thái (stateless authentication) giữa ứng dụng di động và máy chủ web.
- **Giao tiếp client-server qua mạng không ổn định:** Mọi thao tác đều phải đi qua mạng di động, đòi hỏi cơ chế tải dữ liệu bất đồng bộ, caching ảnh và xử lý lỗi mạng — nếu không, ứng dụng sẽ giật và mất phản hồi.
- **Tương thích hệ sinh thái Android:** Ứng dụng phải chạy ổn định trên dải thiết bị Android đa dạng về kích thước màn hình và cấu hình phần cứng, đồng thời tuân thủ vòng đời (lifecycle) chặt chẽ của hệ điều hành.

Đề tài **CineFlow** ra đời từ nhu cầu nghiên cứu và xây dựng một ứng dụng xem phim trực tuyến hiện đại trên nền tảng di động, áp dụng kiến trúc client-server và các kỹ thuật lập

trình Android chuẩn mực để giải quyết các thách thức nêu trên trong điều kiện thực tế. Đây không phải là bài toán học thuật đơn thuần mà là một bài toán kỹ thuật phần mềm có tính ứng dụng cao.

1.1.2 Mục tiêu

Mục tiêu của đề án là xây dựng một ứng dụng xem phim trực tuyến hoàn chỉnh trên nền tảng Android — từ phía client đến phía server — với các chức năng cốt lõi gồm: quản lý tài khoản, khám phá nội dung, xem phim/series, xem video ngắn, theo dõi sự kiện thể thao và quản trị nội dung.

Bên cạnh việc hoàn thiện sản phẩm về mặt chức năng, nhóm tập trung vào việc giải quyết các bài toán kỹ thuật thực tế trong lập trình di động:

- Áp dụng kiến trúc client–server với REST API — Android client giao tiếp với máy chủ Spring Boot qua giao thức HTTP, dữ liệu trao đổi dưới dạng JSON.
- Tổ chức ứng dụng Android theo mô hình phân lớp (Layered Architecture) kết hợp Repository Pattern, tách biệt rõ ràng tầng giao diện, tầng dữ liệu và tầng giao tiếp mạng.
- Xây dựng luồng xác thực không trạng thái dựa trên JSON Web Token, cho phép duy trì phiên đăng nhập bền vững giữa các lần mở ứng dụng và tự động đính kèm chứng chỉ xác thực vào mọi yêu cầu HTTP.
- Tích hợp trình phát video chuyên dụng hỗ trợ định dạng streaming hiện đại, có khả năng phát phim lẻ, series nhiều tập và sự kiện trực tiếp trong cùng một ứng dụng.
- Thiết kế hệ thống phân quyền hai vai trò (người xem và quản trị viên) ngay trong cùng một ứng dụng, với Admin Panel cho phép quản lý nội dung (CRUD phim, danh mục, người dùng) và xem thống kê.

Phạm vi

Đề án bao gồm hai nhóm người dùng: Người xem (User) và Quản trị viên (Admin), triển khai trên nền tảng Android. Ứng dụng cho iOS, phiên bản Web và Smart TV nằm ngoài phạm vi của đề án này.

1.2 Cơ sở lý thuyết

Để giải quyết các bài toán đặt ra, đồ án áp dụng một tập hợp các công nghệ và kiến trúc hiện đại trong lập trình di động và phát triển web. Phần này trình bày nền tảng lý thuyết của từng thành phần và lý do lựa chọn.

1.2.1 Nền tảng Android và ngôn ngữ Java

Android là hệ điều hành di động phổ biến nhất thế giới, được phát triển bởi Google trên nền nhân Linux và máy ảo ART (Android Runtime). Một ứng dụng Android được tổ chức xoay quanh bốn thành phần chính: **Activity** (đại diện cho một màn hình), **Fragment** (một mảnh giao diện có thể tái sử dụng bên trong Activity), **Service** (xử lý nền) và **Broadcast Receiver** (lắng nghe sự kiện hệ thống).

Đồ án sử dụng **Java 11** làm ngôn ngữ lập trình cho phía Android — vừa phù hợp với chương trình môn Lập trình di động, vừa tận dụng được hệ sinh thái thư viện Android Java đã rất trưởng thành. Ứng dụng được biên dịch với `compileSdk 36` và đặt mức `minSdk 24` (Android 7.0 Nougat), bao phủ phần lớn thiết bị Android đang hoạt động.

1.2.2 Kiến trúc Client–Server và REST API

Kiến trúc **Client–Server** phân chia hệ thống thành hai phần: phía khách (client) đảm nhận giao diện và tương tác người dùng, phía máy chủ (server) tập trung dữ liệu và logic nghiệp vụ. Hai phía giao tiếp với nhau qua mạng theo một giao thức thoả thuận trước — phổ biến nhất hiện nay là HTTP/HTTPS.

Trên nền HTTP, **REST (Representational State Transfer)** là phong cách kiến trúc do Roy Fielding đề xuất, mô hình hoá mỗi thực thể nghiệp vụ thành một *tài nguyên* được định danh bằng URL và thao tác qua các phương thức HTTP chuẩn (GET, POST, PUT, DELETE). REST giúp API có cấu trúc dễ đoán, dễ tài liệu hoá và độc lập với ngôn ngữ ở hai phía.

Trong CineFlow, toàn bộ tương tác giữa ứng dụng Android và máy chủ đều đi qua một tập các endpoint REST với tiền tố `/api/v1`, dữ liệu trao đổi dưới định dạng JSON và được bọc trong một bao chuẩn hoá (`success, message, data`) để tầng client xử lý lỗi thống nhất.

1.2.3 Kiến trúc phân lớp và mẫu Repository

Layered Architecture là một trong những mẫu kiến trúc lâu đời và phổ biến nhất: hệ thống được chia thành các tầng (layer) chồng lên nhau, mỗi tầng chỉ phụ thuộc vào tầng ngay bên

dưới. Cách tổ chức này tối đa hoá tính tách rời (separation of concerns) và giúp việc thay đổi một tầng không lan rộng ra toàn bộ hệ thống.

Repository Pattern là phần mở rộng đặc trưng của kiến trúc phân lớp, đóng vai trò là một lớp trung gian *duy nhất* để tầng giao diện truy cập dữ liệu, ẩn đi nguồn gốc thật sự (mạng, cơ sở dữ liệu, cache). Nhờ Repository, các Fragment/Activity trên Android không cần biết dữ liệu đến từ Retrofit hay từ một cache nội bộ — chúng chỉ quan sát kết quả qua một luồng dữ liệu duy nhất.

Trong CineFlow, ứng dụng Android được phân thành các tầng: *UI* (Fragment, Activity, Adapter), *Repository*, *Network* (Retrofit + OkHttp) và *Auth* (quản lý phiên đăng nhập). Phía backend Spring Boot tổ chức tương ứng theo ba tầng cổ điển: *Controller – Service – Repository*.

1.2.4 Vòng đời (Lifecycle) trong Android

Một trong những đặc thù khó nhất của lập trình Android là **vòng đời** của các thành phần giao diện có thể bị hệ điều hành can thiệp bất kỳ lúc nào — khi xoay màn hình, khi chuyển ứng dụng, hoặc khi hệ thống giải phóng bộ nhớ. Một Activity hay Fragment trải qua một chuỗi trạng thái xác định (`onCreate`, `onStart`, `onResume`, `onPause`, `onStop`, `onDestroy`), và mã nguồn của lập trình viên phải tôn trọng chuỗi trạng thái này.

Việc gọi mạng, đăng ký listener hay giữ tham chiếu View không đúng lifecycle là nguyên nhân hàng đầu của hai loại lỗi kinh điển trên Android: *crash* (ví dụ `IllegalStateException` khi cập nhật UI sau khi Fragment đã bị huỷ) và *rò rỉ bộ nhớ* (memory leak) khi callback bất đồng bộ giữ tham chiếu đến Activity đã chết.

CineFlow giải quyết bài toán này bằng cách định nghĩa một lớp trừu tượng chuẩn hoá luồng khởi tạo cho mọi Fragment, đồng thời sử dụng `LiveData` để dữ liệu luôn được phát tới UI một cách *lifecycle-aware* – chỉ cập nhật khi UI thực sự đang hiển thị.

1.2.5 Giao tiếp HTTP với Retrofit và OkHttp

Retrofit là thư viện HTTP client type-safe do Square phát triển, hoạt động theo phong cách *declarative*: lập trình viên khai báo một interface Java với các phương thức được gắn annotation (`@GET`, `@POST`, `@Path`, `@Body`); Retrofit sinh mã runtime để biến mỗi phương thức thành một lời gọi HTTP thực sự. Việc tuần tự hoá và giải tuần tự hoá JSON được uỷ thác cho **Gson**.

Retrofit chạy trên nền **OkHttp** – thư viện HTTP cấp thấp hỗ trợ HTTP/2, connection pooling, cache và đặc biệt là cơ chế **Interceptor** cho phép can thiệp vào mọi request/response. Đồ án tận dụng cơ chế Interceptor để cài đặt một bộ chặn xác thực: trước mỗi request, JWT của người dùng được tự động đính kèm vào header `Authorization` mà tầng nghiệp vụ không

cần can thiệp.

1.2.6 Phát video trực tuyến với ExoPlayer (Media3)

Thành phần `VideoView` mặc định của Android có nhiều hạn chế: không hỗ trợ streaming thích nghi, khó tùy biến giao diện và xử lý lỗi mạng kém. Đồ án lựa chọn **Media3 ExoPlayer** – bộ thư viện chính thức của Google kế thừa và thay thế ExoPlayer truyền thống – cung cấp một trình phát media mở rộng được, hiệu năng cao và phù hợp với các ứng dụng streaming chuyên nghiệp.

ExoPlayer hỗ trợ các định dạng streaming hiện đại như **HLS** (HTTP Live Streaming) và **DASH** (Dynamic Adaptive Streaming over HTTP), tự động điều chỉnh chất lượng video theo băng thông thực tế (adaptive bitrate streaming), đồng thời cho phép tùy biến hoàn toàn giao diện điều khiển. Quan trọng hơn, ExoPlayer được thiết kế để tích hợp chặt chẽ với lifecycle của Activity, giúp giải phóng tài nguyên đúng thời điểm và tránh các lỗi rò rỉ bộ nhớ thường gặp khi phát video trên Android.

1.2.7 Tải và cache ảnh với Glide

Một ứng dụng xem phim có rất nhiều hình ảnh: banner trang chủ, poster phim, thumbnail tập phim, ảnh đại diện kênh thể thao... Tải ảnh trực tiếp trên luồng giao diện sẽ làm ứng dụng giật, đồng thời ảnh độ phân giải cao dễ gây tràn bộ nhớ (`OutOfMemoryError`).

Glide là thư viện tải ảnh chuyên dụng cho Android, cung cấp ba khả năng cốt lõi: (1) tải ảnh bất đồng bộ với placeholder trong khi chờ, (2) cache hai cấp – bộ nhớ và đĩa – nhằm giảm tối đa số lượt tải lặp, (3) tự động cắt và biến đổi ảnh theo kích thước view đích để tránh OOM. Đặc biệt, Glide tự động huỷ tác vụ tải ảnh khi vòng đời của Fragment/Activity kết thúc, giúp tránh rò rỉ bộ nhớ một cách tự nhiên.

1.2.8 Điều hướng với ViewPager2 và BottomNavigationView

Trải nghiệm điển hình của một ứng dụng streaming là các tab nội dung song song mà người dùng có thể chuyển qua lại nhanh chóng. CineFlow áp dụng mô hình **ViewPager2 + BottomNavigationView**. `ViewPager2` quản lý việc cuộn ngang giữa các Fragment với hiệu năng cao nhờ tái sử dụng view, còn `BottomNavigationView` hiển thị ở cạnh dưới làm điểm điều hướng cố định – vừa trực quan, vừa tuân thủ chuẩn Material Design.

Bên cạnh luồng tab chính, các luồng phụ (đăng nhập, đăng ký, chi tiết phim, phát video, quản trị) được triển khai bằng các Activity riêng và mở qua Intent – phù hợp với

mô hình điều hướng nông (shallow navigation) thường thấy ở các ứng dụng giải trí.

1.2.9 Hệ thống thiết kế Material Design

Material Design là hệ thống thiết kế do Google phát triển, hiện đã tới phiên bản thứ ba (Material 3). Hệ thống này cung cấp một bộ quy tắc nhất quán về màu sắc, kiểu chữ, khoảng cách, độ nâng (elevation) và chuyển động, giúp các ứng dụng Android có ngôn ngữ hình ảnh thống nhất và quen thuộc với người dùng.

CineFlow sử dụng chủ đề tối (dark theme) – phù hợp với trải nghiệm xem phim – và tổ chức màu sắc theo **design token**: các nhóm *surface* (mặt nền), *brand* (thương hiệu), *text* (chữ) và *status* (trạng thái) được khai báo trong `colors.xml` với tên có ngữ nghĩa rõ ràng. Quy ước trong toàn bộ dự án là layout XML luôn tham chiếu màu qua `@color/<tên>` thay vì mã hex cứng, đảm bảo có thể thay đổi theme tập trung mà không phải sửa hàng trăm file.

1.2.10 Phát triển backend với Spring Boot

Spring Boot là một framework phát triển ứng dụng Java do Pivotal (nay là VMware) phát triển, dựa trên Spring Framework. Spring Boot loại bỏ phần lớn cấu hình XML lặp lại của Spring truyền thống bằng cơ chế *auto-configuration* – framework tự động suy ra cấu hình hợp lý dựa trên các thư viện có trên classpath – và *embedded server* (Tomcat đóng gói sẵn) cho phép chạy ứng dụng web bằng một dòng lệnh duy nhất.

Phía backend của CineFlow được xây dựng trên Spring Boot 3.5 (Java 17), tổ chức theo kiến trúc phân lớp ba tầng truyền thống: **Controller** (mở endpoint REST, validate dữ liệu vào), **Service** (chứa logic nghiệp vụ và transaction) và **Repository** (truy cập cơ sở dữ liệu qua Spring Data JPA). Bảo mật được uỷ thác cho mô-đun Spring Security.

1.2.11 Cơ sở dữ liệu PostgreSQL và quản lý schema với Flyway

PostgreSQL được lựa chọn làm hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) chính cho backend. Lý do bao gồm: tính tuân thủ ACID nghiêm ngặt giúp đảm bảo toàn vẹn dữ liệu, hỗ trợ kiểu dữ liệu phong phú (UUID, JSONB, mảng), khả năng mở rộng tốt và là mã nguồn mở hoàn toàn.

Trong môi trường phát triển nhiều người, một thách thức luôn xuất hiện là: làm sao đảm bảo schema cơ sở dữ liệu của tất cả thành viên luôn đồng bộ với mã nguồn? **Flyway** giải quyết vấn đề này bằng cách quản lý các bản di trú (migration) theo phiên bản. Mỗi thay đổi schema được lưu thành một file SQL có đánh số (`v1`, `v2`, `v3`...); khi ứng dụng khởi động,

Flyway đối chiếu với bảng lịch sử `flyway_schema_history` và tự động áp dụng tuần tự các bản chưa chạy. Nhờ đó, schema luôn được kiểm soát giống như mã nguồn – có lịch sử, có thể review và có thể tái lập lại trên mọi môi trường.

1.2.12 Xác thực không trạng thái với JSON Web Token

JSON Web Token (JWT) là một tiêu chuẩn mở (RFC 7519) cho phép truyền thông tin xác thực dưới dạng một chuỗi gọn, đã được ký số. Một JWT gồm ba phần được phân tách bởi dấu chấm: `header.payload.signature`. Phần *payload* chứa các thông tin nhận dạng người dùng (claims) như `userId`, `role`; phần *signature* được sinh bằng thuật toán băm có khoá bí mật, đảm bảo token không bị giả mạo.

Ưu điểm cốt lõi của JWT là tính **không trạng thái** (stateless): máy chủ chỉ cần khoá bí mật để xác thực, không cần lưu phiên (session) trong bộ nhớ hay cơ sở dữ liệu – rất phù hợp với REST API và các ứng dụng di động phải duy trì đăng nhập dài hạn.

Trong CineFlow, sau khi đăng nhập thành công, máy chủ trả về một JWT có thời hạn 2 giờ; ứng dụng Android lưu token này vào bộ nhớ riêng và đính kèm vào header `Authorization` ở mọi request tiếp theo. Mật khẩu người dùng ở phía máy chủ được băm bằng **BCrypt** – thuật toán băm chậm có salt nội tại, chống lại các tấn công brute-force và rainbow table.

CHƯƠNG 2

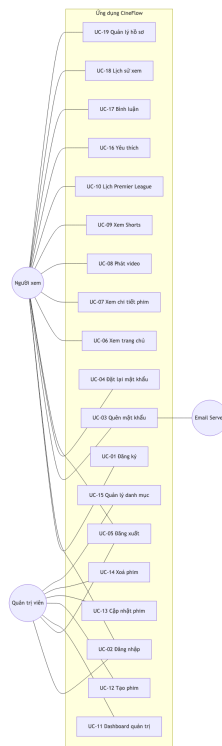
THIẾT KẾ HỆ THỐNG

2.1 Phân tích yêu cầu hệ thống

2.1.1 Yêu cầu chức năng (Functional Requirements)

Sơ đồ Use Case Tổng thể (Overview Use Case Diagram)

Sơ đồ dưới đây minh họa các nhóm nghiệp vụ chính của ứng dụng CineFlow, được phân bổ cho hai nhóm Tác nhân (Actors) cốt lõi: Người xem (User) và Quản trị viên (Admin). Hệ thống Email Server tham gia gián tiếp trong luồng khôi phục mật khẩu.



Hình 2.1. Sơ đồ Use Case Tổng quát của ứng dụng CineFlow

Đặc tả Use Case Chi tiết

Dưới đây là đặc tả chi tiết 19 luồng nghiệp vụ cốt lõi (Use Cases) của hệ thống CineFlow, được ánh xạ trực tiếp từ các endpoint của REST API và chia thành 5 nhóm chức năng (Features)

độc lập.

Nhóm 1: Xác thực và Tài khoản

Mã Usecase	UC-01
Tên Usecase	Đăng ký tài khoản
Tác nhân (Actors)	Người xem (User)
Mô tả ngắn	Người dùng tạo tài khoản mới trên CineFlow bằng email và mật khẩu.
Điều kiện tiên quyết	Người dùng chưa đăng nhập, có kết nối Internet.
Điều kiện đảm bảo	Tài khoản được tạo với vai trò <code>ROLE_USER</code> , mật khẩu được băm trước khi lưu.
Luồng sự kiện chính	1. Người dùng nhập <code>username</code>, <code>email</code>, <code>password</code>. 2. Hệ thống kiểm tra trùng lặp email/username. 3. Hệ thống băm mật khẩu bằng <code>BCrypt</code> và lưu vào <code>CSDL</code>.
Luồng rẽ nhánh	Email đã tồn tại: Hệ thống trả lỗi <i>Email already exists</i> .
Yêu cầu chức năng (FR)	◇ FR_01.1: Mật khẩu phải có tối thiểu 6 ký tự. ◇ FR_01.2: Mật khẩu không được lưu dưới dạng plaintext.

Mã Usecase	UC-02
Tên Usecase	Đăng nhập hệ thống
Tác nhân (Actors)	Người xem (User), Quản trị viên (Admin)
Mô tả ngắn	Người dùng đăng nhập bằng username và mật khẩu để nhận JWT.
Điều kiện tiên quyết	Tài khoản đã được đăng ký.
Điều kiện đảm bảo	Nhận về JWT có hạn 2 giờ; thông tin phiên được lưu vào <code>SharedPreferences</code> .
Luồng sự kiện chính	1. Người dùng nhập username/mật khẩu. 2. Hệ thống đối chiếu mật khẩu băm trong CSDL. 3. Hệ thống sinh JWT chứa <code>userId</code> và <code>role</code>, trả về cho client.
Luồng rẽ nhánh	Sai thông tin: Báo lỗi <i>Invalid username or password</i> .
Yêu cầu chức năng (FR)	◇ FR_02.1: JWT phải được ký bằng <code>JWT_SECRET</code> từ biến môi trường. ◇ FR_02.2: Token có thời hạn 2 giờ.

Mã Usecase	UC-03
Tên Usecase	Quên mật khẩu
Tác nhân (Actors)	Người xem (User), Email Server
Mô tả ngắn	Người dùng yêu cầu khôi phục mật khẩu qua email.
Điều kiện tiên quyết	Không yêu cầu đăng nhập.
Điều kiện đảm bảo	Hệ thống sinh <code>resetToken</code> có thời hạn và gửi link khôi phục vào email.
Luồng sự kiện chính	1. Người dùng nhập email. 2. Hệ thống tạo <code>resetToken</code> ngẫu nhiên. 3. Hệ thống gửi email chứa link kèm token qua SMTP.
Luồng rẽ nhánh	Email không tồn tại: Trả về 200 OK nhưng không gửi mail để tránh lộ thông tin tài khoản.
Yêu cầu chức năng (FR)	◇ FR_03.1: <code>resetToken</code> chỉ có hiệu lực một lần duy nhất trong 30 phút.

Mã Usecase	UC-04
Tên Usecase	Đặt lại mật khẩu
Tác nhân (Actors)	Người xem (User)
Mô tả ngắn	Người dùng đặt mật khẩu mới sau khi xác thực qua <code>resetToken</code> .
Điều kiện tiên quyết	Sở hữu <code>resetToken</code> hợp lệ, chưa hết hạn.
Điều kiện đảm bảo	Mật khẩu mới được băm và lưu; token bị vô hiệu hoá.
Luồng sự kiện chính	1. Client kiểm tra token còn hiệu lực qua <code>GET /auth/validate-reset-token</code>. 2. Người dùng nhập mật khẩu mới. 3. Hệ thống băm mật khẩu, cập nhật CSDL, vô hiệu hoá token.
Luồng rẽ nhánh	Token hết hạn hoặc đã dùng: Báo lỗi <i>Reset token expired or already used</i> .
Yêu cầu chức năng (FR)	◊ FR_04.1: Sau khi đặt lại, mọi phiên đăng nhập cũ vẫn hợp lệ cho đến khi JWT hết hạn tự nhiên.

Mã Usecase	UC-05
Tên Usecase	Đăng xuất
Tác nhân (Actors)	Người xem (User), Quản trị viên (Admin)
Mô tả ngắn	Người dùng đăng xuất khỏi ứng dụng.
Điều kiện tiên quyết	Người dùng đã đăng nhập.
Điều kiện đảm bảo	Thông tin phiên trong <code>SharedPreferences</code> bị xoá; UI quay về trạng thái khách.
Luồng sự kiện chính	1. Người dùng bấm <i>Đăng xuất</i> trong tab More. 2. <code>AuthManager</code> xoá token, <code>userId</code>, <code>username</code>, <code>role</code>. 3. UI <code>MoreFragment</code> tự cập nhật về trạng thái chưa đăng nhập.
Luồng rẽ nhánh	Không có.
Yêu cầu chức năng (FR)	◊ FR_05.1: Đăng xuất chỉ là thao tác phía client; backend là stateless.

Nhóm 2: Khám phá Nội dung

Mã Usecase	UC-06
Tên Usecase	Xem trang chủ
Tác nhân (Actors)	Người xem (User), Khách (Guest)
Mô tả ngắn	Người dùng xem trang chủ với nhiều khu vực nội dung (banner, phim mới, sự kiện thể thao, series, phim trong ngày).
Điều kiện tiên quyết	Không yêu cầu đăng nhập.
Điều kiện đảm bảo	Trang chủ hiển thị đầy đủ năm khu vực nội dung.
Luồng sự kiện chính	1. HomeFragment gọi GET /films/home. 2. Backend trả về object chứa banners, newReleases, sportEvents, hotSeries, dailyMovies. 3. Client render từng khu vực vào RecyclerView tương ứng.
Luồng rẽ nhánh	Mất kết nối: Hiển thị thông báo lỗi và nút <i>Thử lại</i> .
Yêu cầu chức năng (FR)	◊ FR_06.1: Trang chủ phải hiển thị trong ≤ 2 giây với cache ảnh Glide.

Mã Usecase	UC-07
Tên Usecase	Xem chi tiết phim
Tác nhân (Actors)	Người xem (User)
Mô tả ngắn	Người dùng xem mô tả chi tiết, trailer và danh sách tập của một phim.
Điều kiện tiên quyết	Người dùng đã chọn một phim từ trang chủ hoặc kết quả tìm kiếm.
Điều kiện đảm bảo	FilmDetailActivity hiển thị đầy đủ thông tin phim và danh sách Episode.
Luồng sự kiện chính	1. Client gọi GET /films/{id}. 2. Backend trả về Film kèm danh sách Episode. 3. UI render mô tả, ảnh thumbnail và danh sách tập.
Luồng rẽ nhánh	Không tìm thấy phim: Backend trả 404, client hiển thị thông báo <i>Phim không tồn tại</i> .
Yêu cầu chức năng (FR)	◇ FR_07.1: Với phim lẻ (SINGLE), hệ thống tự sinh một Episode mặc định để tái sử dụng luồng phát.

Mã Usecase	UC-08
Tên Usecase	Phát video
Tác nhân (Actors)	Người xem (User)
Mô tả ngắn	Người dùng phát phim lẻ, một tập của series, hoặc sự kiện trực tiếp.
Điều kiện tiên quyết	Đã chọn một Episode/phim cụ thể, có kết nối Internet.
Điều kiện đảm bảo	Video bắt đầu phát trong <code>PlayerActivity</code> .
Luồng sự kiện chính	1. Client mở <code>PlayerActivity</code> với <code>videoUrl</code>. 2. <code>ExoPlayer</code> khởi tạo <code>MediaItem</code> và bắt đầu prepare. 3. Player phát video, hiển thị thanh điều khiển.
Luồng rẽ nhánh	URL không khả dụng: Player phát ra <code>Player.Listener.onPlayerError</code> , UI hiển thị lỗi và nút <i>Thử lại</i> .
Yêu cầu chức năng (FR)	◊ FR_08.1: Player phải giải phóng tài nguyên trong <code>onDestroy()</code> để tránh rò rỉ bộ nhớ.

Mã Usecase	UC-09
Tên Usecase	Xem video ngắn (Shorts)
Tác nhân (Actors)	Người xem (User)
Mô tả ngắn	Người dùng cuộn dọc xem các video ngắn, tự động phát/dừng theo vị trí cuộn.
Điều kiện tiên quyết	Có kết nối Internet.
Điều kiện đảm bảo	Mỗi lần cuộn chỉ có duy nhất một video đang phát; các video còn lại dừng.
Luồng sự kiện chính	1. <code>ShortsFragment</code> gọi <code>GET /films/shorts</code>. 2. Client gắn danh sách vào <code>ViewPager2</code> đọc. 3. Khi page change, video đang hiển thị tự play, video trước/sau pause.
Luồng rẽ nhánh	Hết danh sách: Hiển thị placeholder <i>Đã hết shorts</i> .
Yêu cầu chức năng (FR)	◊ FR_09.1: Bộ nhớ <code>ExoPlayer</code> phải được tái sử dụng để tránh tạo mới mỗi lần cuộn.

Mã Usecase	UC-10
Tên Usecase	Xem lịch Premier League
Tác nhân (Actors)	Người xem (User)
Mô tả ngắn	Người dùng xem lịch thi đấu Ngoại hạng Anh, có thể lọc theo mùa giải và theo ngày.
Điều kiện tiên quyết	Có kết nối Internet.
Điều kiện đảm bảo	Lịch hiển thị theo giờ Việt Nam, đã quy đổi từ UTC.
Luồng sự kiện chính	1. PremierLeagueFragment chọn mùa giải. 2. PremierLeagueRepository gọi backend với tham số season, date. 3. Backend trả về danh sách trận; client render theo nhóm ngày.
Luồng rẽ nhánh	Không có trận nào: Hiển thị thông báo <i>Chưa có trận đấu trong ngày.</i>
Yêu cầu chức năng (FR)	◇ FR_10.1: Mọi mốc thời gian phải được hiển thị theo múi giờ Asia/Ho_Chi_Minh.

Nhóm 3: Quản trị Phim

Mã Usecase	UC-11
Tên Usecase	Xem Dashboard quản trị
Tác nhân (Actors)	Quản trị viên (Admin)
Mô tả ngắn	Admin truy cập bảng điều khiển với các chỉ số tổng quan và liên kết tới các khu vực quản lý.
Điều kiện tiên quyết	Tài khoản có vai trò <code>ROLE_ADMIN</code> .
Điều kiện đảm bảo	<code>AdminDashboardActivity</code> hiển thị các thẻ <code>Films/Users/Categories/Stats</code> .
Luồng sự kiện chính	1. <code>MoreFragment</code> kiểm tra <code>AuthManager.isAdmin()</code>. 2. Hiển thị card <i>Admin Panel</i>. 3. Mở <code>AdminDashboardActivity</code> qua <code>Intent</code>.
Luồng rẽ nhánh	User thường: Card Admin Panel bị ẩn, không thể truy cập.
Yêu cầu chức năng (FR)	◊ FR_11.1: Việc phân quyền ở client chỉ là tầng UI; backend phải kiểm tra lại role ở mọi endpoint admin.

Mã Usecase	UC-12
Tên Usecase	Tạo phim mới
Tác nhân (Actors)	Quản trị viên (Admin)
Mô tả ngắn	Admin tạo một phim mới với đầy đủ các thuộc tính: tiêu đề, mô tả, thumbnail, trailer, loại phim.
Điều kiện tiên quyết	Đã đăng nhập với vai trò Admin.
Điều kiện đảm bảo	Phim mới được lưu vào CSDL và xuất hiện trong AdminFilmListActivity.
Luồng sự kiện chính	- Admin bấm FAB <i>Tạo mới</i> trong AdminFilmListActivity. - FilmFormDialogFragment mở, Admin điền form. - Client gọi POST /admin/films. - Danh sách phim tự refresh.
Luồng rẽ nhánh	Thiếu trường bắt buộc: Backend trả <i>Validation Failed (400)</i> ; client hiển thị lỗi tại field tương ứng.
Yêu cầu chức năng (FR)	◇ FR_12.1: Trường <code>title</code> và <code>type</code> là bắt buộc.

Mã Usecase	UC-13
Tên Usecase	Cập nhật phim
Tác nhân (Actors)	Quản trị viên (Admin)
Mô tả ngắn	Admin sửa thông tin một phim đã có.
Điều kiện tiên quyết	Phim cần sửa tồn tại trong CSDL.
Điều kiện đảm bảo	Phim được cập nhật, ứng dụng người dùng nhìn thấy ngay ở lần gọi API tiếp theo.
Luồng sự kiện chính	1. Admin bấm item phim trong danh sách → chọn <i>Sửa</i>. 2. <code>FilmFormDialogFragment</code> mở với dữ liệu hiện tại. 3. Admin sửa và bấm <i>Lưu</i>. 4. Client gọi <code>PUT /admin/films/{id}</code>.
Luồng rẽ nhánh	Phim không tồn tại: Backend trả 404; client hiển thị thông báo và đóng dialog.
Yêu cầu chức năng (FR)	◇ FR_13.1: Các tập (Episode) của phim không bị xoá khi sửa thông tin phim.

Mã Usecase	UC-14
Tên Usecase	Xoá phim
Tác nhân (Actors)	Quản trị viên (Admin)
Mô tả ngắn	Admin xoá một phim khỏi hệ thống.
Điều kiện tiên quyết	Phim cần xoá tồn tại.
Điều kiện đảm bảo	Phim và các Episode liên quan bị xoá khỏi CSDL theo cơ chế cascade.
Luồng sự kiện chính	1. Admin bấm icon xoá trên một item phim. 2. Dialog xác nhận hiển thị. 3. Admin xác nhận; client gọi <code>DELETE /admin/films/{id}</code>. 4. Danh sách tự refresh.
Luồng rẽ nhánh	Phim đang được tham chiếu: Backend trả lỗi ràng buộc khoá ngoại; client hiển thị thông báo.
Yêu cầu chức năng (FR)	◇ FR_14.1: Thao tác xoá là không thể hoàn tác.

Nhóm 4: Quản lý Danh mục

Mã Usecase	UC-15
Tên Usecase	Quản lý danh mục thể loại
Tác nhân (Actors)	Quản trị viên (Admin)
Mô tả ngắn	Admin thêm, sửa, xoá các danh mục (Category) để phân loại phim.
Điều kiện tiên quyết	Đã đăng nhập với vai trò Admin.
Điều kiện đảm bảo	Danh sách <code>Category</code> được cập nhật và phản ánh trên trang chủ ở lần gọi tiếp theo.
Luồng sự kiện chính	- Admin mở khu vực <i>Categories</i> từ Dashboard. - Thực hiện thao tác CRUD trên danh sách. - Client gọi endpoint <code>/admin/categories</code> tương ứng.
Luồng rẽ nhánh	Xoá danh mục đang có phim: Backend chặn xoá và trả lỗi; Admin phải gỡ liên kết phim trước.
Yêu cầu chức năng (FR)	◇ FR_15.1: Tên danh mục là duy nhất trong toàn hệ thống.

Nhóm 5: Tương tác và Cá nhân hóa

Mã Usecase	UC-16
Tên Usecase	Đánh dấu phim yêu thích
Tác nhân (Actors)	Người xem (User)
Mô tả ngắn	Người dùng thêm hoặc xóa một bộ phim khỏi danh sách yêu thích cá nhân.
Điều kiện tiên quyết	Người dùng đã đăng nhập vào hệ thống.
Điều kiện đảm bảo	Trạng thái yêu thích được lưu trữ đồng bộ dưới cơ sở dữ liệu và hiển thị đúng trên UI.
Luồng sự kiện chính	1. Người dùng bấm nút "Thêm vào yêu thích" trên màn hình chi tiết phim. 2. Client gửi yêu cầu <code>POST /favorites/{filmId}</code> hoặc <code>DELETE /favorites/{filmId}</code>. 3. Hệ thống cập nhật bảng <code>favorites</code> trong CSDL và phản hồi thành công. 4. UI cập nhật màu sắc và văn bản của nút tương ứng.
Luồng rẽ nhánh	Chưa đăng nhập: Hệ thống chuyển hướng người dùng tới màn hình đăng nhập.
Yêu cầu chức năng (FR)	◇ FR_16.1: Phải hiển thị đúng danh sách phim yêu thích cá nhân tại tab Favorites.

Mã Usecase	UC-17
Tên Usecase	Bình luận phim và video ngắn
Tác nhân (Actors)	Người xem (User)
Mô tả ngắn	Người dùng viết và gửi bình luận dưới các phim hoặc video ngắn, cũng như xem các bình luận khác.
Điều kiện tiên quyết	Có kết nối Internet; Đã đăng nhập để gửi bình luận.
Điều kiện đảm bảo	Bình luận mới được hiển thị ngay lập tức ở đầu danh sách bình luận.
Luồng sự kiện chính	1. Người dùng cuộn tới phần bình luận, hệ thống gọi <code>GET /films/{filmId}/comments</code>. 2. Người dùng nhập văn bản và nhấn gửi. 3. Client gọi <code>POST /films/{filmId}/comments</code>. 4. Hệ thống lưu bình luận vào bảng <code>film_comments</code> và thêm vào giao diện.
Luồng rẽ nhánh	Chưa đăng nhập: Ấn ô nhập nội dung bình luận, hiển thị liên kết đăng nhập.
Yêu cầu chức năng (FR)	◇ FR_17.1: Nội dung bình luận không được để trống. ◇ FR_17.2: Người dùng chỉ có quyền xóa bình luận do chính mình tạo ra.

Mã Usecase	UC-18
Tên Usecase	Lưu lịch sử và tiến trình xem
Tác nhân (Actors)	Người xem (User)
Mô tả ngắn	Hệ thống tự động ghi nhận lại tiến trình phát video (vị trí giây cuối cùng) của người dùng để tiếp tục phát ở lần sau.
Điều kiện tiên quyết	Người dùng đang phát một tập phim.
Điều kiện đảm bảo	Tiến trình được cập nhật định kỳ lên backend và lưu trữ tại bảng <code>watch_history</code> .
Luồng sự kiện chính	1. Khi người dùng phát video, ExoPlayer ghi nhận thời điểm dừng phát (hoặc gửi định kỳ). 2. Client gọi <code>PUT /watch-history/{episodeId}</code> kèm vị trí giây. 3. Hệ thống lưu/cập nhật thông tin vào bảng <code>watch_history</code>. 4. Ở lần tiếp theo người dùng mở lại phim này, hệ thống gợi ý tiếp tục phát từ điểm đã dừng.
Luồng rẽ nhánh	Không có.
Yêu cầu chức năng (FR)	◊ FR_18.1: Vị trí tiến trình lưu phải đảm bảo không vượt quá tổng thời lượng của tập phim.

Mã Usecase	UC-19
Tên Usecase	Quản lý hồ sơ người dùng
Tác nhân (Actors)	Người xem (User)
Mô tả ngắn	Người dùng xem thông tin tài khoản cá nhân, đổi mật khẩu hoặc cập nhật họ tên.
Điều kiện tiên quyết	Người dùng đã đăng nhập.
Điều kiện đảm bảo	Thông tin cập nhật thành công được đồng bộ vào bảng <code>users</code> và cập nhật trực tiếp trên giao diện Client.
Luồng sự kiện chính	1. Người dùng mở tab More, chọn sửa hồ sơ. 2. Hệ thống hiển thị thông tin lấy từ <code>GET /user/me</code>. 3. Người dùng cập nhật họ tên hoặc đổi mật khẩu và nhấn Lưu. 4. Client gọi <code>PUT /user/profile</code> hoặc <code>POST /user/change-password</code> để đồng bộ dữ liệu.
Luồng rẽ nhánh	Đổi mật khẩu sai mật khẩu cũ: Backend trả lỗi xác thực mật khẩu; client hiển thị thông báo lỗi.
Yêu cầu chức năng (FR)	◇ FR_19.1: Mật khẩu mới thay đổi phải tuân thủ độ dài tối thiểu 6 ký tự.

2.1.2 Yêu cầu phi chức năng (Non-Functional Requirements)

Bảng 2.1 tổng hợp các yêu cầu phi chức năng của hệ thống CineFlow, chia theo 6 thuộc tính chất lượng.

Bảng 2.1. Yêu cầu phi chức năng của hệ thống CineFlow

Thuộc tính	Yêu cầu	Giải pháp kỹ thuật
Hiệu năng	Thời gian phản hồi API $\leq 500\text{ms}$ (p95) cho các endpoint nghiệp vụ chính. Video bắt đầu phát trong ≤ 3 giây kể từ khi người dùng bấm Play.	Backend tách Repository – Service – Controller; Android client tải bắt đồng bộ qua Volley + Gson (phối hợp OkHttp3Stack); ExoPlayer với HLS adaptive bitrate.
Khả năng mở rộng	Ứng dụng có thể bổ sung loại nội dung mới (Series, Live, Shorts) mà không phải sửa đổi cấu trúc cốt lõi.	Mô hình dữ liệu thống nhất quanh Film với enum <code>FilmType</code> ; backend phân lớp ba tầng giúp thêm endpoint mới mà không phá vỡ tầng dữ liệu.
Tính nhất quán dữ liệu	Schema cơ sở dữ liệu của tất cả thành viên trong nhóm phải luôn đồng bộ. Các thao tác CRUD trên Admin Panel phản ánh ngay xuống ứng dụng người dùng.	Flyway Migrations quản lý schema theo phiên bản. Backend đọc/ghi cùng một PostgreSQL nên không có khoảng trễ đồng bộ.
Bảo mật	Mọi endpoint quản trị phải xác thực. Mật khẩu không được lưu dưới dạng plaintext. Token đăng nhập phải có hạn sử dụng.	JWT Bearer Token với chữ ký HMAC-SHA256, thời hạn 2 giờ. Mật khẩu băm bằng BCrypt. Android client lưu token trong <code>SharedPreferences</code> ở chế độ private.
Tính sẵn sàng	Ứng dụng phải xử lý được trường hợp mất mạng tạm thời mà không bị treo (ANR – Application Not Responding).	Mọi lời gọi mạng dùng Volley <code>enqueue</code> bắt đồng bộ trên luồng nền (qua lớp bọc <code>Call</code> tương thích); UI cập nhật qua <code>LiveData</code> lifecycle-aware; hiển thị trạng thái lỗi thân thiện khi mất kết nối.
Bảo trì & vận hành	Schema database chỉ được thay đổi qua <code>28</code> migration, không sửa trực tiếp. Mã nguồn Android tuân thủ kiến trúc phân lớp, dễ kiểm thử.	Flyway cho phía backend; <code>BaseFragment</code> chuẩn hoá lifecycle Android; tất cả <code>Fragment</code> tuân theo luồng <code>getLayoutId</code> →

2.2 Thiết kế lược đồ dữ liệu

Khác với các hệ thống Microservices theo mô hình Database-per-Service, CineFlow là một ứng dụng đơn khối (Monolith) ở phía backend nên toàn bộ dữ liệu được lưu tập trung trong một cơ sở dữ liệu PostgreSQL duy nhất. Lược đồ dữ liệu được tổ chức quanh ba nhóm thực thể chính: *Tài khoản & Gói cước*, *Nội dung phim* và *Tương tác người dùng*.

2.2.1 Nhóm Tài khoản và Gói cước

- **User (Aggregate Root):** Lưu trữ thông tin tài khoản cốt lõi như `username`, `email`, `password` (đã băm BCrypt) và `role`. Trường `role` sử dụng kiểu `enum UserRole (ROLE_USER, ROLE_ADMIN)` để phân biệt hai nhóm người dùng ngay trong cùng một bảng – thiết kế đơn giản nhưng đủ cho phạm vi đồ án.
- **SubscriptionPackage:** Danh mục các gói thuê bao (Catalog) mà ứng dụng cung cấp – ví dụ gói cơ bản, gói cao cấp với các đặc quyền khác nhau.
- **UserSubscription:** Bảng quan hệ N-N giữa `User` và `SubscriptionPackage`, lưu lại trạng thái thuê bao (`SubscriptionStatus – Active, Expired...`), ngày bắt đầu và ngày hết hạn. Thiết kế này cho phép một người dùng có nhiều phiên đăng ký theo thời gian.
- **RefreshToken:** Lưu trữ token làm mới phiên đăng nhập cho mỗi người dùng (`user_id`), bao gồm chuỗi `token` băm ngẫu nhiên, ngày hết hạn `expiryDate` và cờ vô hiệu hóa `revoked`. Thiết kế này cho phép duy trì và làm mới phiên đăng nhập của người dùng một cách an toàn mà không cần yêu cầu đăng nhập lại bằng mật khẩu nhiều lần.

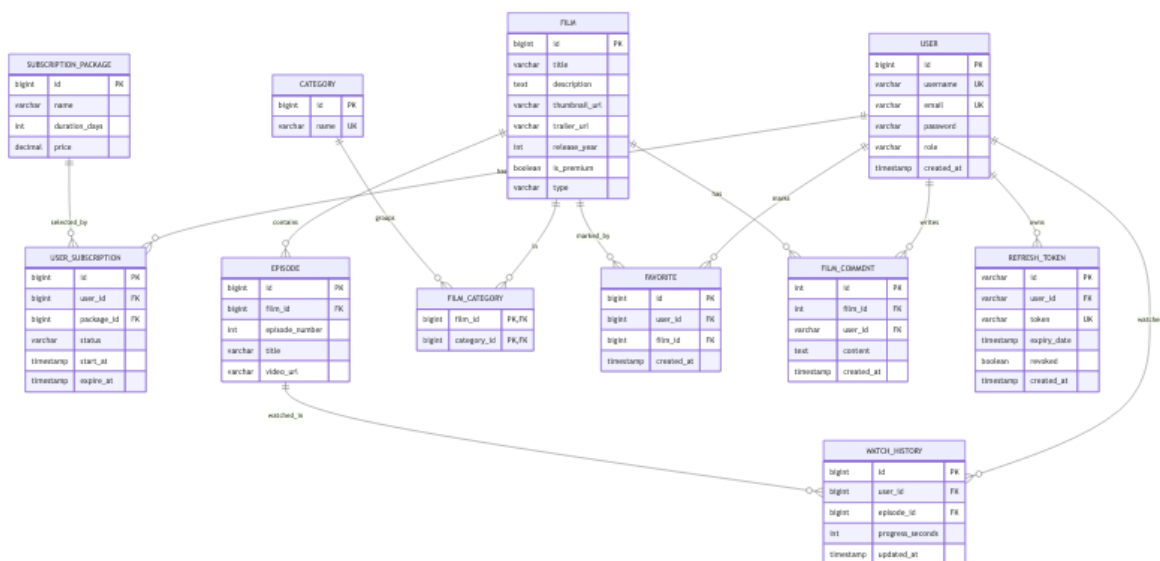
2.2.2 Nhóm Nội dung phim

- **Film (Aggregate Root):** Lưu trữ thông tin gốc của một phim – bao gồm `title`, `description`, `thumbnailUrl`, `trailerUrl`, `releaseYear`, `isPremium` và đặc biệt là `type (FilmType: SINGLE, SERIES, LIVE)`. Việc dùng chung một thực thể `Film` cho cả ba loại nội dung giúp đơn giản hoá đáng kể API: trang chủ, danh sách yêu thích, lịch sử xem đều thao tác trên cùng một kiểu dữ liệu.
- **Episode:** Quản lý từng tập của một bộ phim nhiều tập. Mỗi `Episode` liên kết tới đúng một `Film` qua khoá ngoại `film_id`, đồng thời lưu `episodeNumber`, `title` và `videoUrl` riêng. Với phim lẻ (`SINGLE`), hệ thống tự sinh một `Episode` mặc định để tái sử dụng cùng một luồng phát video.

- **Category:** Danh mục thể loại (Hành động, Kinh dị, HÀi hước...). `FilmCategory` là bảng trung gian giải quyết quan hệ N-N giữa `Film` và `Category`, cho phép một phim thuộc nhiều thể loại đồng thời.

2.2.3 Nhóm Tương tác người dùng

- **Favorite:** Bảng trung gian N-N giữa `User` và `Film` ghi lại các phim mà người dùng đã đánh dấu yêu thích. Trường `createdAt` cho phép sắp xếp danh sách theo thứ tự thêm gần nhất.
- **WatchHistory:** Lưu lại lịch sử xem của người dùng – không chỉ là việc đã xem hay chưa mà còn lưu `progressSeconds` (vị trí dừng) để hỗ trợ tính năng *Resume Watching*, cho phép tiếp tục xem từ điểm dừng trên cùng một thiết bị hoặc thiết bị khác.
- **FilmComment:** Lưu trữ các bình luận của người dùng đối với các phim hoặc video ngắn. Thực thể này liên kết tới cả bảng `User` (`user_id`) và bảng `Film` (`film_id`), chứa trường `content` lưu nội dung bình luận và `createdAt` để sắp xếp hiển thị theo trình tự thời gian.



Hình 2.2. Sơ đồ Thực thể Liên kết (ERD) của hệ thống CineFlow

2.2.4 Quản lý phiên bản schema với Flyway

Toàn bộ thay đổi schema được mã hoá thành các file SQL có đánh số trong thư mục `src/main/resources` của backend. Hiện tại đồ án có 14 bản di trú được áp dụng tuần tự:

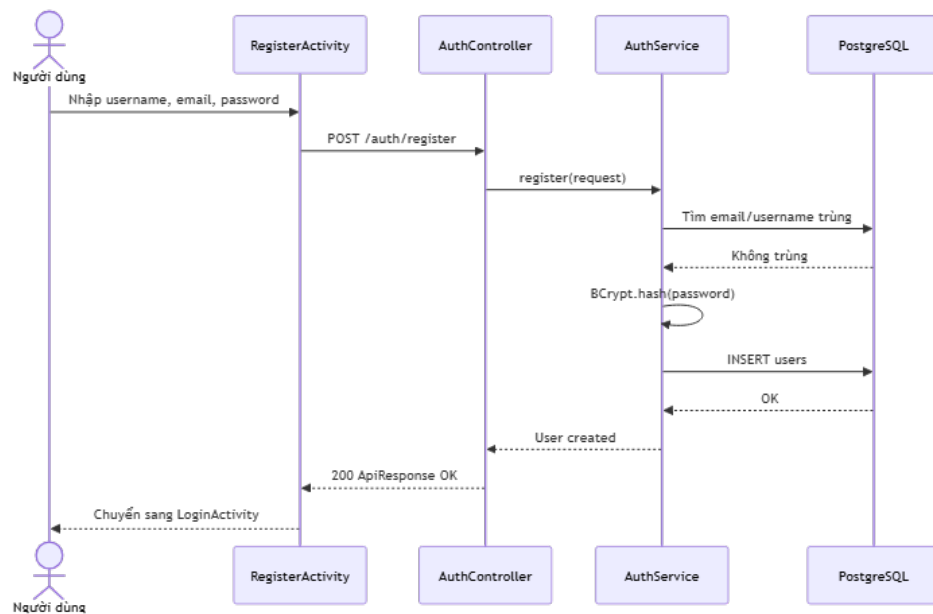
- **V1__create_tables.sql:** Khởi tạo cấu trúc các bảng dữ liệu cốt lõi (users, categories, films, film_categories, episodes, packages, user_subscriptions, favorites, watch_history).
- **V2__convert_pg_enum_columns_to_varchar.sql:** Chuyển các kiểu enum gốc của PostgreSQL sang VARCHAR để tăng cường khả năng tương thích với JDBC và JPA.
- **V3, V4__update_video_urls.sql, V4__fix_video_urls.sql:** Cập nhật và sửa lỗi các đường dẫn URL video mẫu.
- **V5__add_user_content_constraints.sql:** Bổ sung ràng buộc cho các tương tác của người dùng.
- **V9__align_films_badge_and_remove_legacy_football_tables.sql:** Đồng bộ hóa trường badge của thực thể Film và dọn dẹp các bảng bóng đá cũ không còn sử dụng.
- **V10__seed_premier_league_contents.sql:** Nạp dữ liệu trận đấu và lịch thi đấu Ngoại hạng Anh.
- **V11__create_refresh_tokens_table.sql:** Tạo bảng refresh_tokens phục vụ cho cơ chế làm mới JWT.
- **V12__fix_admin_password_hash.sql:** Sửa lại chuỗi băm mật khẩu tài khoản quản trị viên mặc định.
- **V13__seed_realistic_movie_dataset.sql:** Nạp danh sách phim điện ảnh thực tế có chất lượng và độ dài thực (>1 giờ).
- **V14__create_film_comments_table.sql:** Tạo bảng film_comments hỗ trợ tính năng bình luận phim và shorts.
- **V15__insert_default_users.sql:** Nạp một số tài khoản người dùng mẫu cho môi trường phát triển.
- **V16, V17__seed_shorts_and_update_media.sql, V17__update_shorts_metadata.sql:** Nạp dữ liệu video ngắn (shorts) và cập nhật siêu dữ liệu tệp tin tương ứng.

Khi backend khởi động, Flyway tự đối chiếu bảng flyway_schema_history với danh sách migration và áp dụng tuần tự các bản chưa chạy, đảm bảo mọi môi trường (máy của thành viên, máy CI, máy demo) luôn có cùng một trạng thái schema.

2.3 Sơ đồ Tuần tự Nghiệp vụ (Sequence Diagrams)

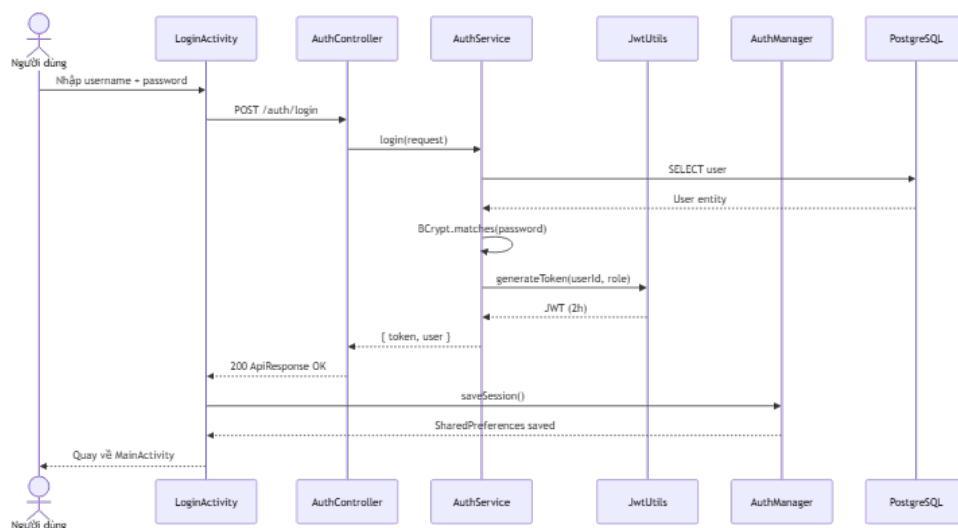
Phần này trình bày luồng tương tác dữ liệu (Data Flow) của các Use Case quan trọng trong hệ thống theo mô hình Boundary – Control – Entity (BCE). Các sơ đồ tập trung vào tầng phân tích, làm rõ vai trò của từng thành phần (Android UI, Repository, Volley/Network, Controller, Service, Repository JPA, Database).

1. UC-01: Đăng ký tài khoản



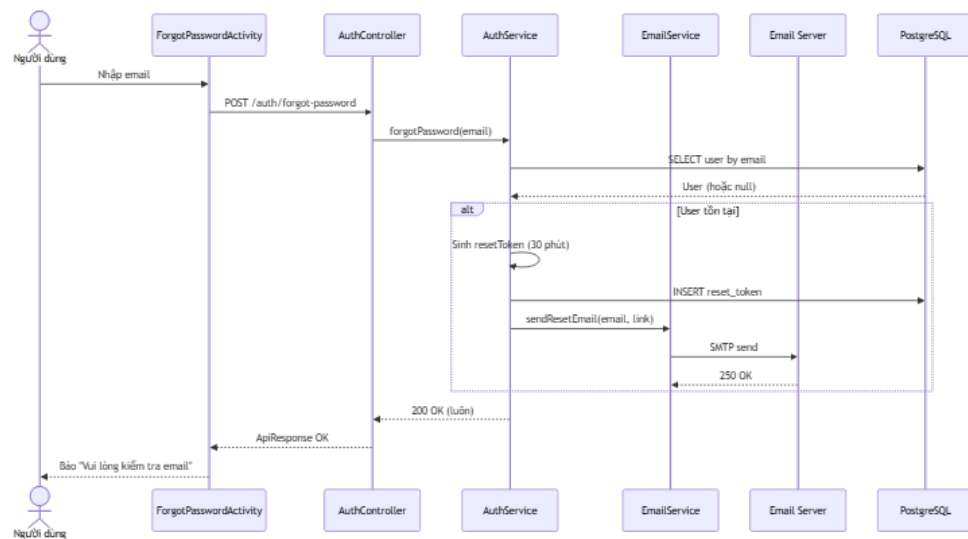
Hình 2.3. Sơ đồ Tuần tự: UC-01 Đăng ký tài khoản

2. UC-02: Đăng nhập hệ thống



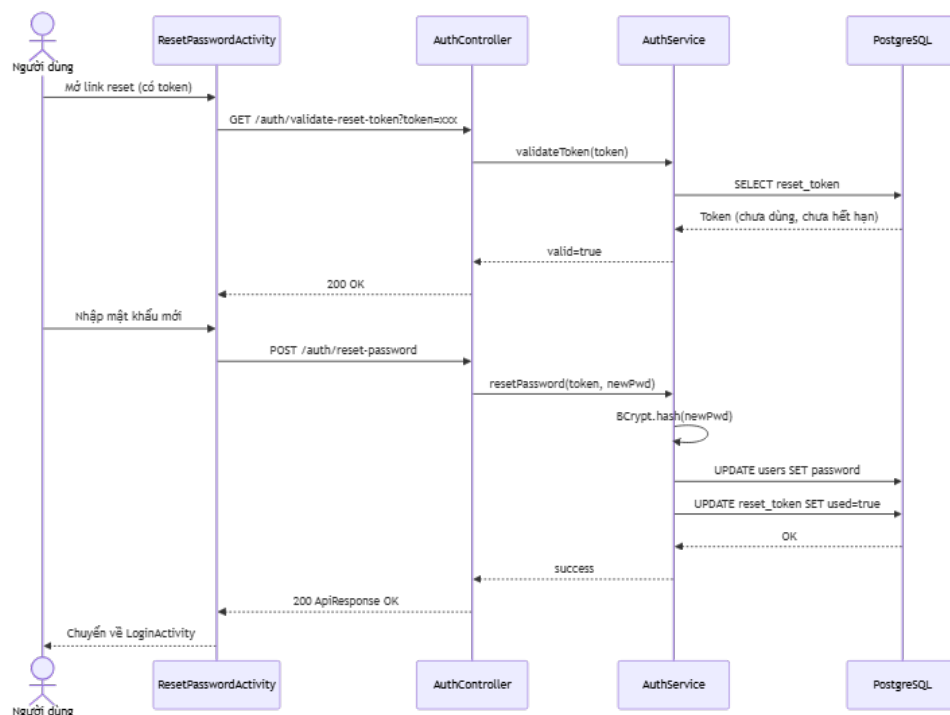
Hình 2.4. Sơ đồ Tuần tự: UC-02 Đăng nhập hệ thống

3. UC-03: Quên mật khẩu



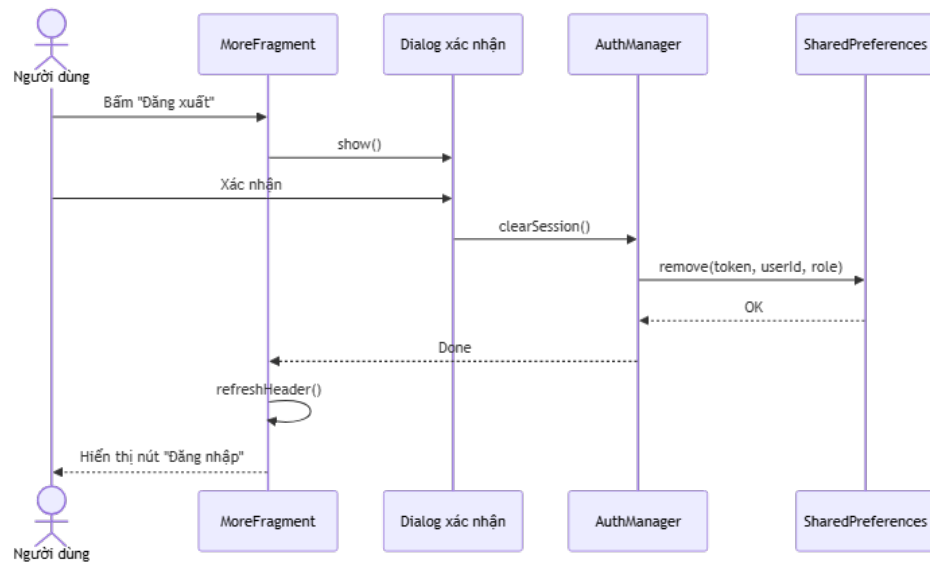
Hình 2.5. Sơ đồ Tuần tự: UC-03 Quên mật khẩu

4. UC-04: Đặt lại mật khẩu



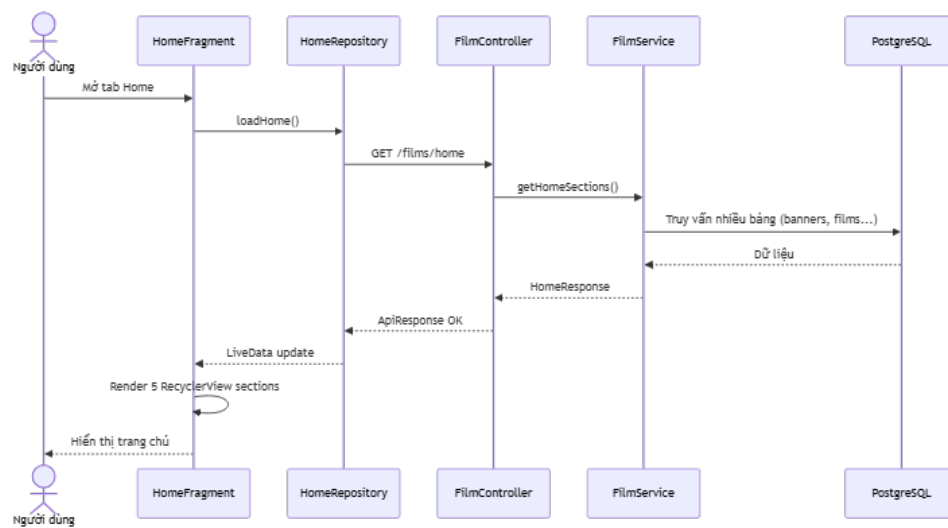
Hình 2.6. Sơ đồ Tuần tự: UC-04 Đặt lại mật khẩu

5. UC-05: Đăng xuất



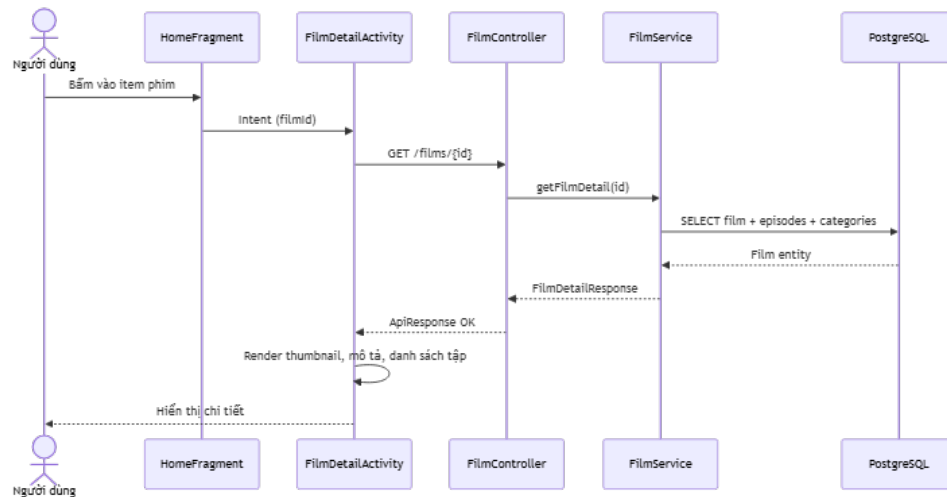
Hình 2.7. Sơ đồ Tuần tự: UC-05 Đăng xuất

6. UC-06: Xem trang chủ



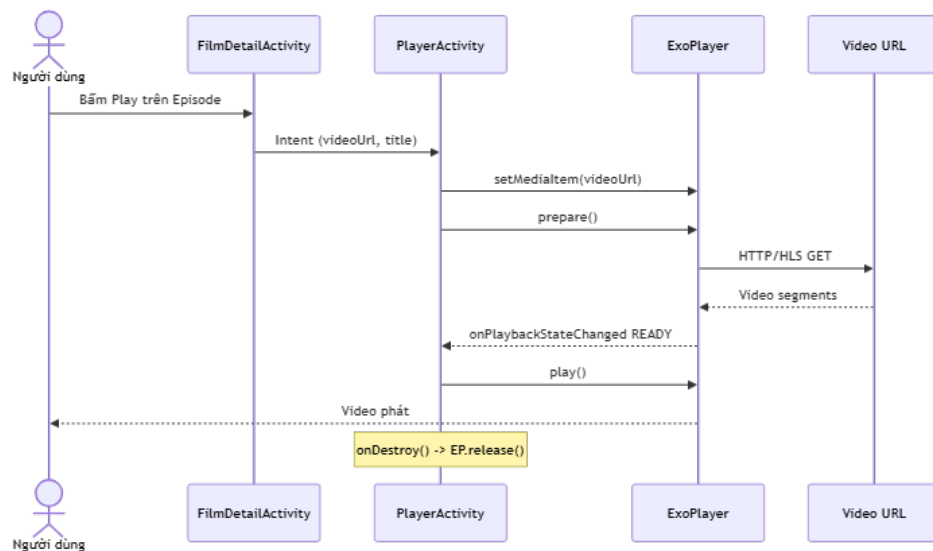
Hình 2.8. Sơ đồ Tuần tự: UC-06 Xem trang chủ

7. UC-07: Xem chi tiết phim



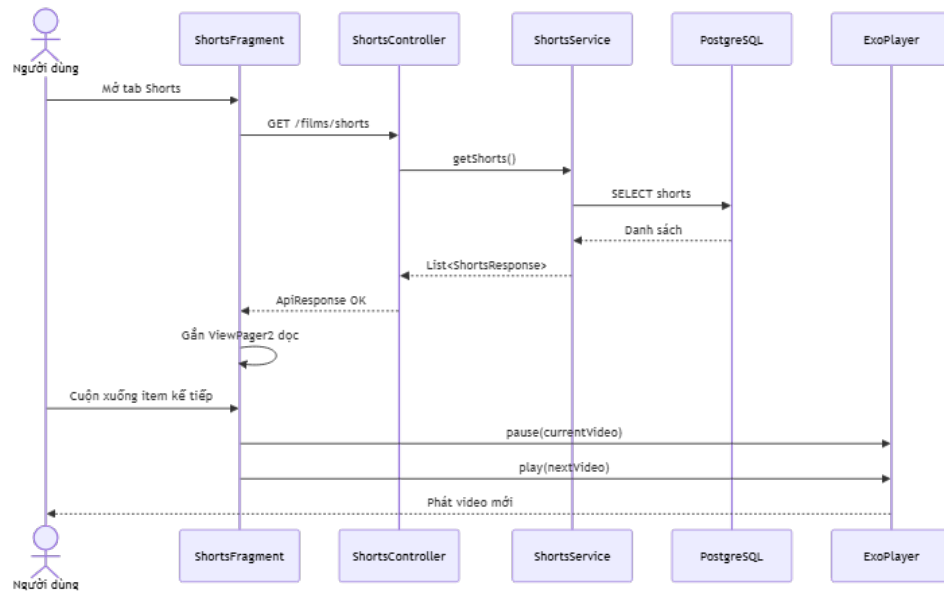
Hình 2.9. Sơ đồ Tuần tự: UC-07 Xem chi tiết phim

8. UC-08: Phát video



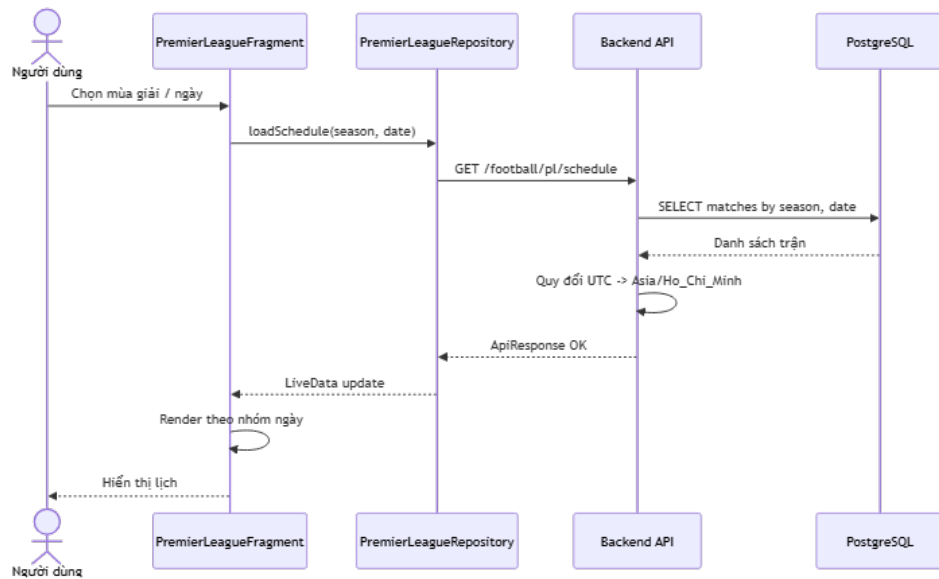
Hình 2.10. Sơ đồ Tuần tự: UC-08 Phát video

9. UC-09: Xem video ngắn (Shorts)



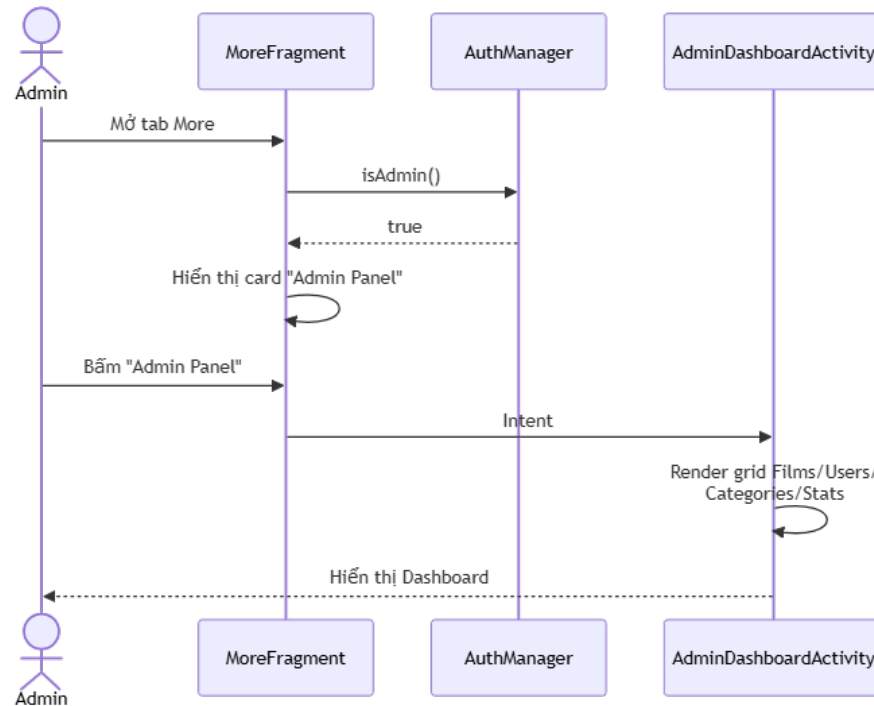
Hình 2.11. Sơ đồ Tuần tự: UC-09 Xem video ngắn

10. UC-10: Xem lịch Premier League



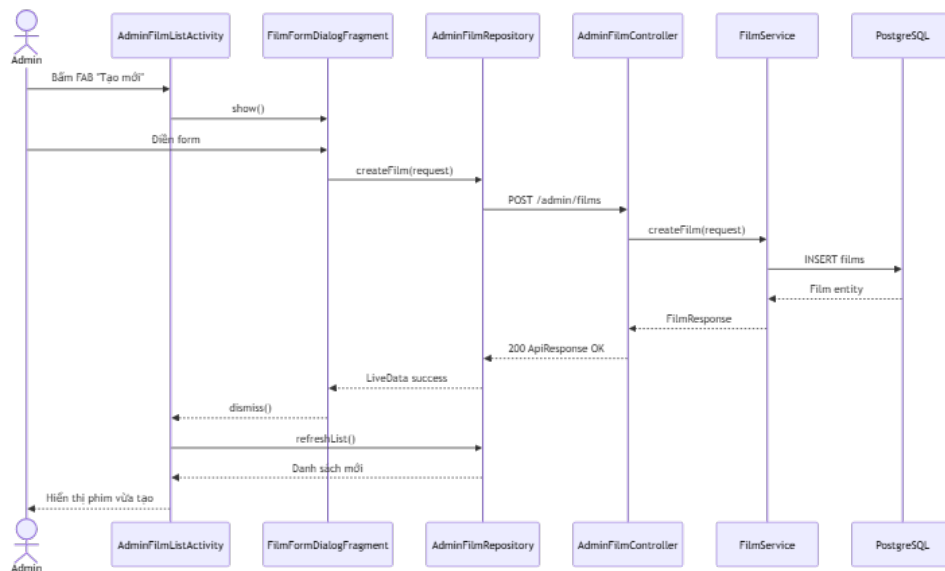
Hình 2.12. Sơ đồ Tuần tự: UC-10 Xem lịch Premier League

11. UC-11: Xem Dashboard quản trị



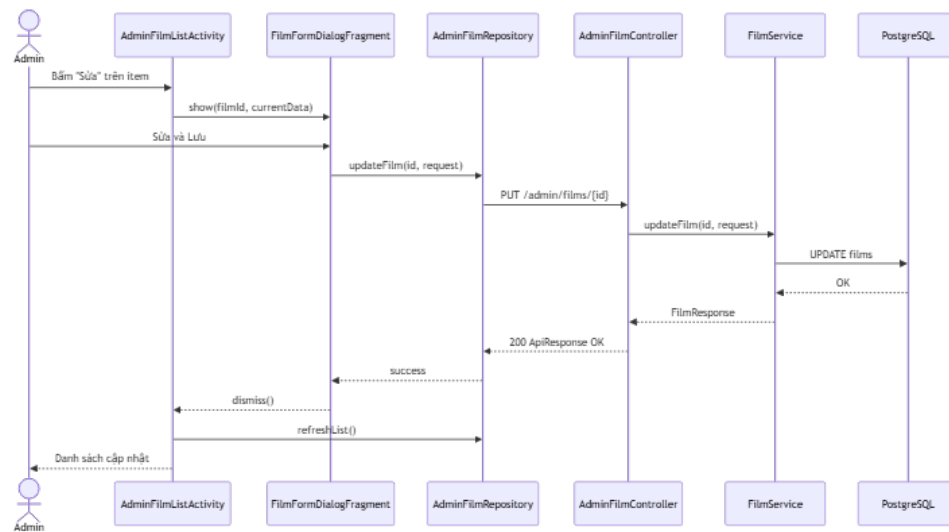
Hình 2.13. Sơ đồ Tuần tự: UC-11 Xem Dashboard quản trị

12. UC-12: Tạo phim mới



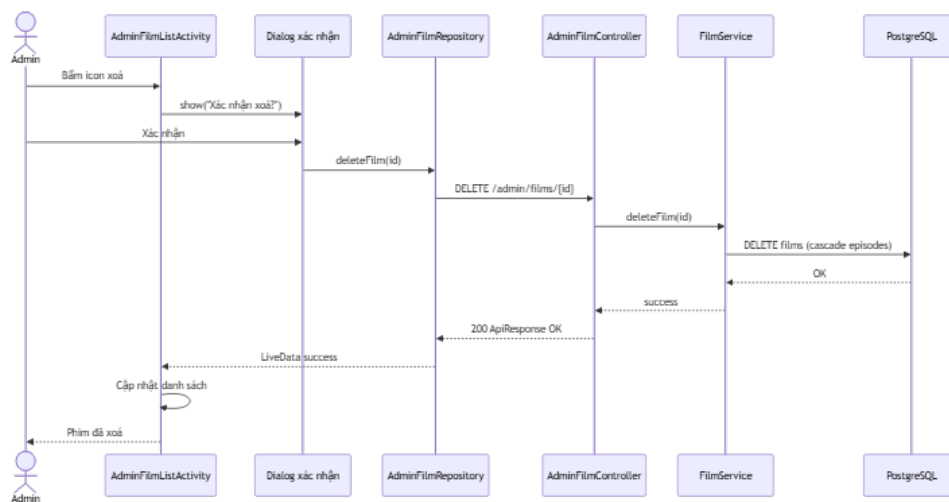
Hình 2.14. Sơ đồ Tuần tự: UC-12 Tạo phim mới

13. UC-13: Cập nhật phim



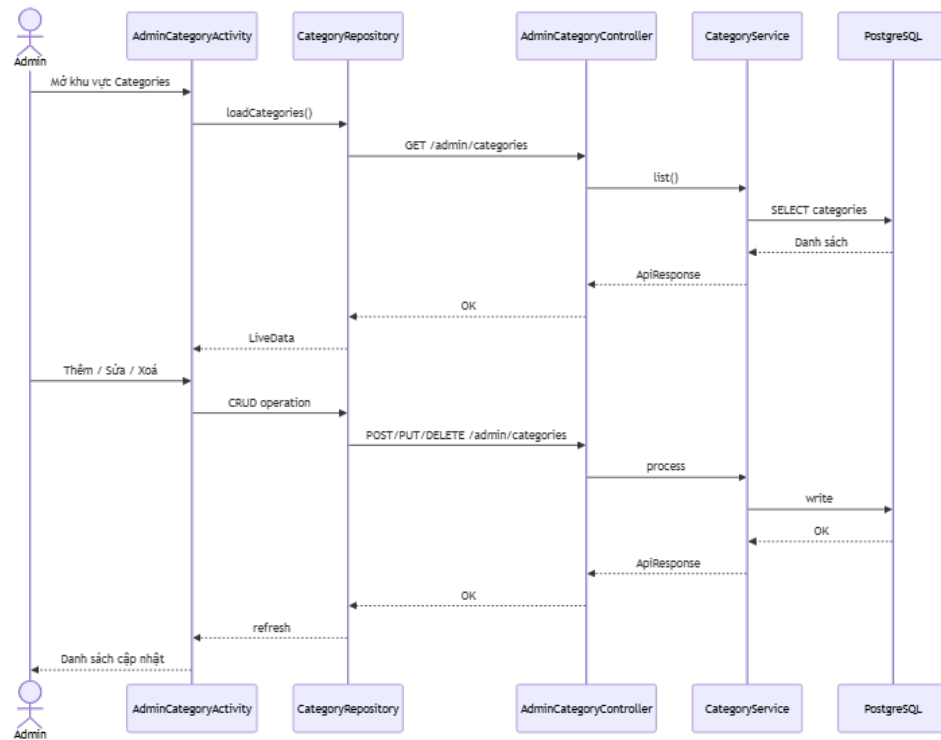
Hình 2.15. Sơ đồ Tuần tự: UC-13 Cập nhật phim

14. UC-14: Xóa phim



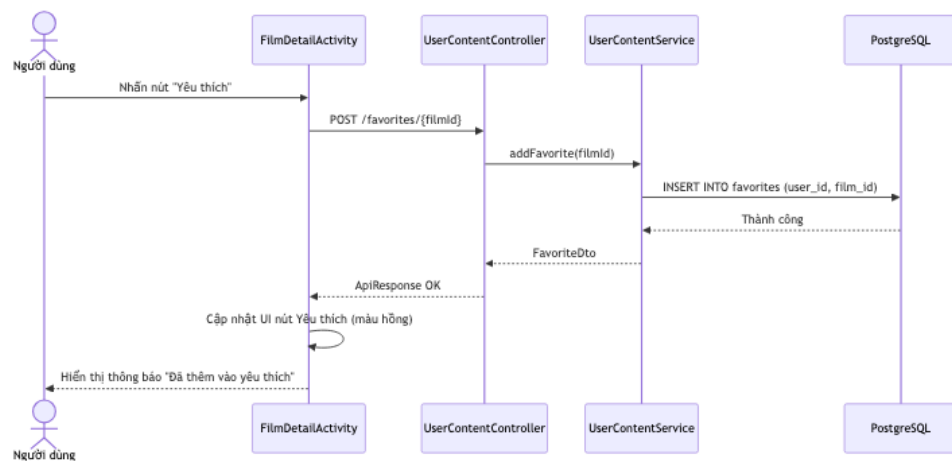
Hình 2.16. Sơ đồ Tuần tự: UC-14 Xóa phim

15. UC-15: Quản lý danh mục thể loại



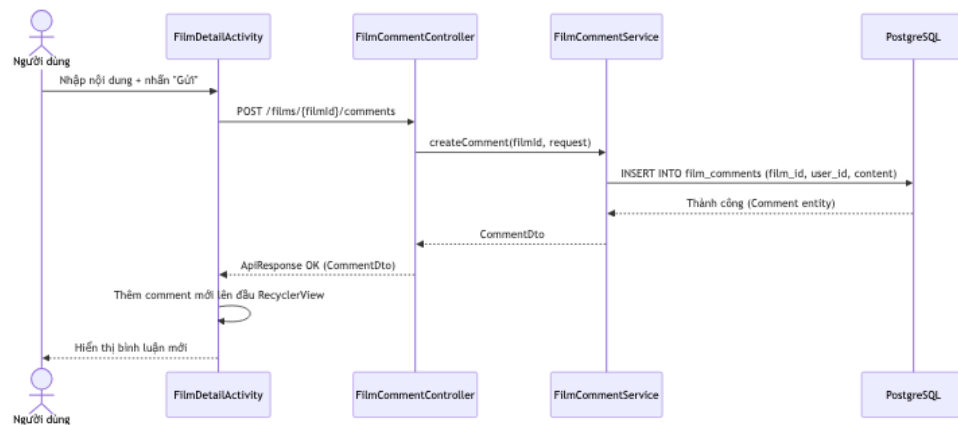
Hình 2.17. Sơ đồ Tuần tự: UC-15 Quản lý danh mục thể loại

16. UC-16: Đánh dấu phim yêu thích



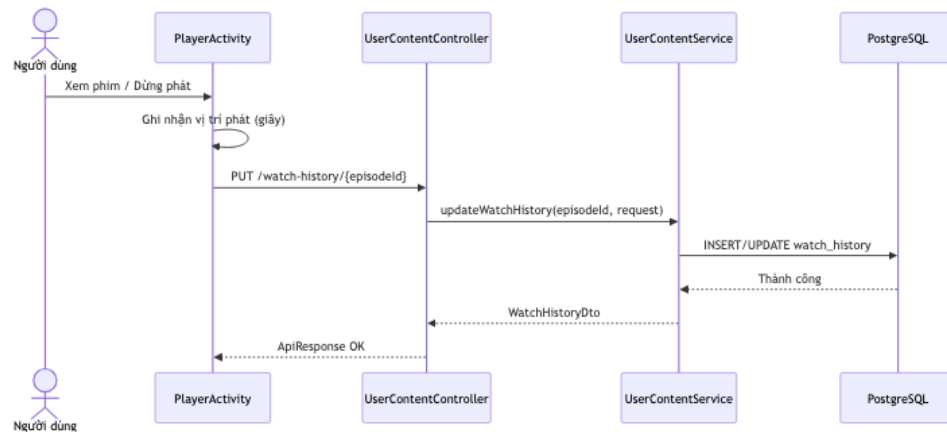
Hình 2.18. Sơ đồ Tuần tự: UC-16 Đánh dấu phim yêu thích

17. UC-17: Bình luận phim và video ngắn



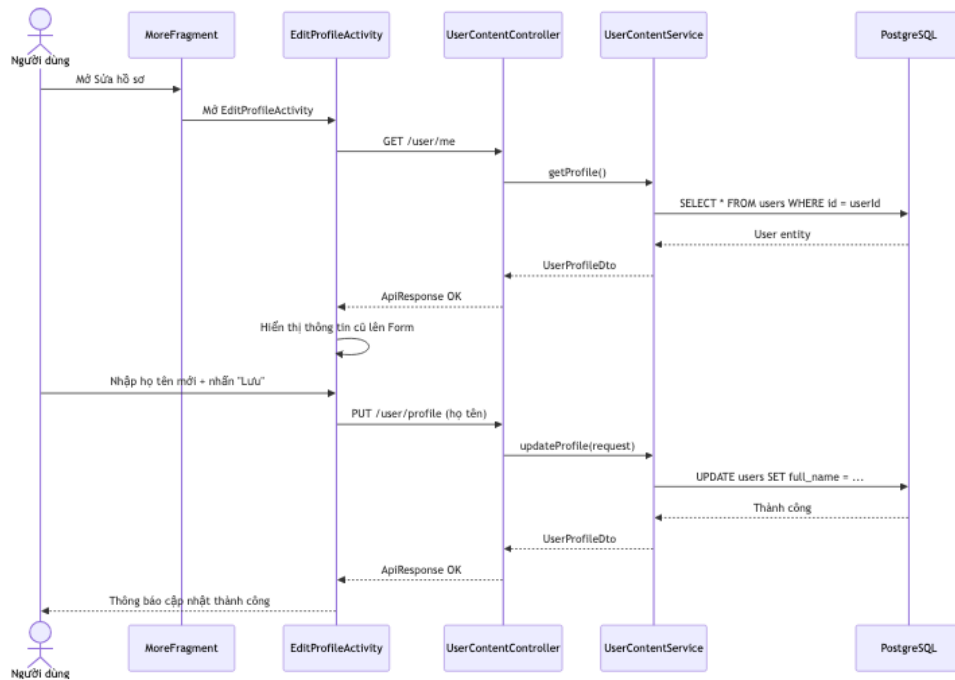
Hình 2.19. Sơ đồ Tuần tự: UC-17 Bình luận phim và video ngắn

18. UC-18: Lưu lịch sử và tiến trình xem



Hình 2.20. Sơ đồ Tuần tự: UC-18 Lưu lịch sử và tiến trình xem

19. UC-19: Quản lý hồ sơ người dùng



Hình 2.21. Sơ đồ Tuần tự: UC-19 Quản lý hồ sơ người dùng

2.4 Kiến trúc tổng thể

2.4.1 Sơ đồ Kiến trúc Tổng thể

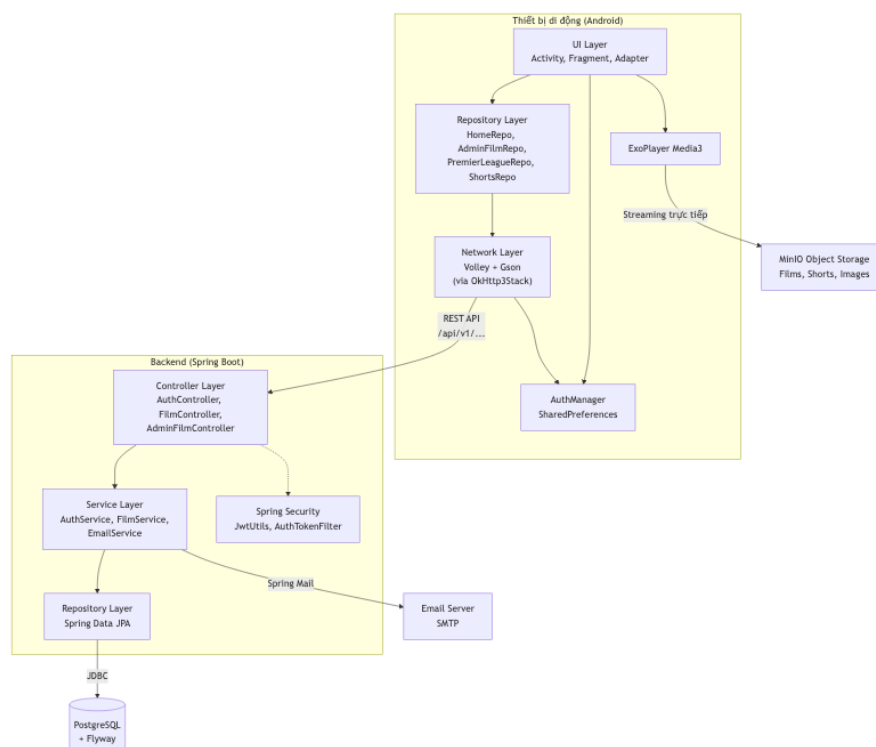
Kiến trúc của hệ thống CineFlow được thiết kế theo mô hình **Client–Server** truyền thống nhưng có phân lớp chặt chẽ ở cả hai phía. Việc lựa chọn kiến trúc đơn khối thay vì Microservices xuất phát từ thực tế của dự án:

- Phạm vi nghiệp vụ của một ứng dụng xem phim ở mức môn học không đủ phức tạp để biện minh cho chi phí vận hành của Microservices (nhiều service, nhiều database, message broker, observability...).
- Đội ngũ phát triển là sinh viên, ưu tiên một mô hình dễ triển khai, dễ debug và dễ trình bày trong báo cáo.

Hệ thống được chia thành các thành phần cốt lõi như sau:

1. **Android Client:** Ứng dụng native viết bằng Java, giao tiếp với backend qua REST API thông qua thư viện Volley (được bọc giao diện tương đương Retrofit).
2. **Spring Boot Backend:** Tổ chức theo ba tầng cổ điển *Controller – Service – Repository*, cung cấp REST API với tiền tố `/api/v1`.

3. **PostgreSQL Database:** Lưu toàn bộ dữ liệu nghiệp vụ. Schema được quản lý bằng Flyway.
4. **Email Server (SMTP):** Đóng vai trò ngoại vi cho luồng khôi phục mật khẩu, được backend gọi qua Spring Mail.
5. **Media Storage (MinIO Object Storage):** Hệ thống sử dụng dịch vụ lưu trữ đối tượng MinIO (self-hosted trên Docker) để quản lý tất cả các tệp tin đa phương tiện (video, ảnh thumbnail). Client truyền phát (streaming) video và tải ảnh trực tiếp từ MinIO thông qua đường dẫn public URL.



Hình 2.22. Sơ đồ Kiến trúc Tổng thể của hệ thống CineFlow

Sơ đồ trên (Hình 2.18) minh họa rõ ràng sự phân chia ba luồng giao tiếp: luồng REST API đồng bộ từ client tới backend, luồng truy vấn JDBC từ backend tới PostgreSQL, và luồng SMTP từ backend tới email server. Phía Android client tự phân lớp UI – Repository – Network, đảm bảo các Fragment/Activity không bao giờ gọi trực tiếp thư viện mạng Volley/Retrofit.

2.4.2 Cơ chế giao tiếp Client–Server

Để đảm bảo ứng dụng phản hồi mượt mà ngay cả khi mạng di động không ổn định, toàn bộ luồng giao tiếp được chia làm ba cơ chế thiết kế chuyên biệt:

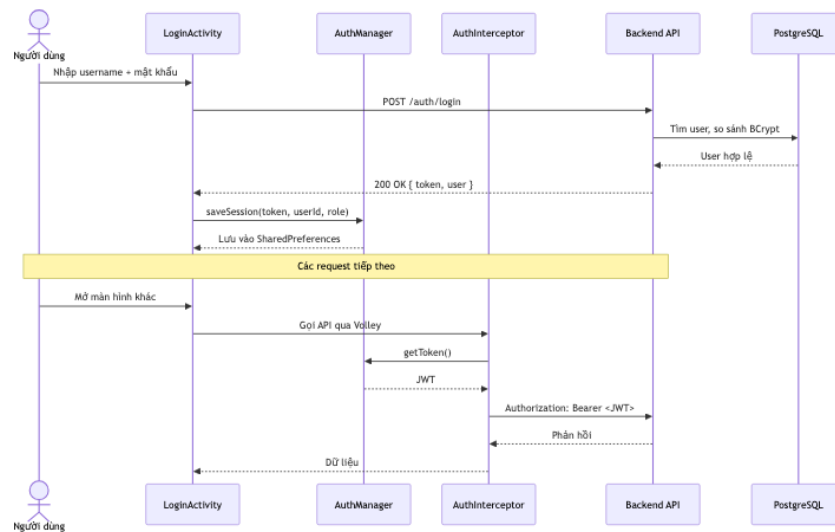
- **Giao tiếp Đồng bộ HTTP/REST:** Mọi thao tác đọc/ghi nghiệp vụ đều đi qua một tập endpoint REST với tiền tố `/api/v1`. Android client sử dụng thư viện Volley để gửi yêu cầu, kết hợp cùng Gson để tự động phân tích cú pháp JSON thành DTO. Để giữ mã nguồn sạch và tương thích kiến trúc, client xây dựng một lớp bọc mỏng cung cấp interface `Call<T>` và `Callback<T>` có cú pháp gọi tương tự Retrofit. Lời gọi API chạy bất đồng bộ qua luồng nền của Volley (`Call.enqueue()`).
- **Tự động xác thực và Làm mới Token (Interceptor & Authenticator):** Đồ án tích hợp thư viện OkHttp làm tầng truyền tải bên dưới của Volley thông qua lớp `OkHttp3Stack`. Cơ chế xác thực sử dụng hai thành phần:
 - `AuthInterceptor`: Đính kèm access token vào header `Authorization: Bearer <token>` của mọi request.
 - `TokenAuthenticator`: Khi server phản hồi mã lỗi 401 `Unauthorized` (access token hết hạn), authenticator sẽ tự động thực hiện cuộc gọi mạng đồng bộ gửi refresh token lên endpoint `/auth/refresh` để lấy cập token mới, lưu lại vào `SharedPreferences` thông qua `AuthManager`, rồi gửi lại request bị lỗi ban đầu một cách trong suốt với người dùng.
- **Phát Video Streaming từ MinIO:** Khác với dữ liệu nghiệp vụ, luồng video được phát trực tiếp từ máy chủ MinIO Object Storage. Khi người dùng bấm Play, `ExoPlayer` truyền tải trực tiếp qua giao thức HTTP từ URL của MinIO (đã được phân quyền public read-only). Thiết kế này giải phóng Spring Boot backend khỏi nhiệm vụ truyền tải dữ liệu media nặng.

Bản đồ Giao tiếp Cụ thể trong các Luồng nghiệp vụ

Để làm rõ cơ chế trên, dưới đây là bản đồ giao tiếp thực tế của hai luồng nghiệp vụ tiêu biểu của ứng dụng CineFlow.

1. Luồng Xác thực và Duy trì Phiên đăng nhập

Luồng này phục vụ việc xác thực người dùng và duy trì phiên trong các phiên làm việc tiếp theo.

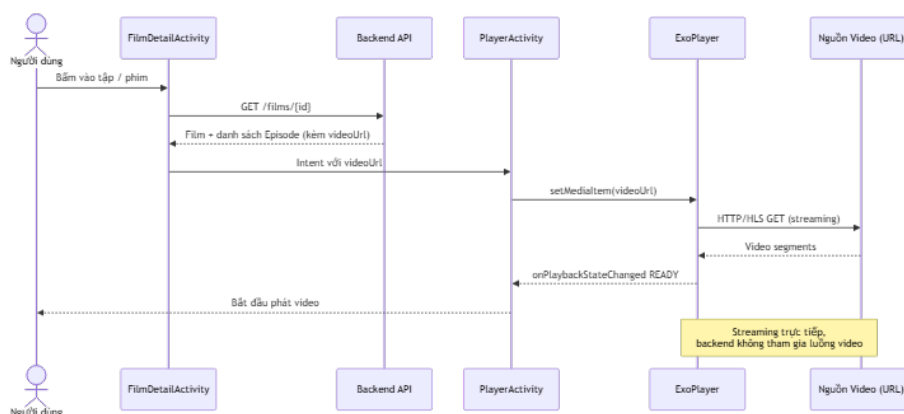


Hình 2.23. Sơ đồ Tuần tự minh họa luồng đăng nhập và đính kèm JWT

Khi người dùng đăng nhập thành công, backend trả về cặp access token (JWT có thời hạn ngắn) và refresh token (hạn dài). Android client lưu cặp token cùng thông tin user vào `SharedPreferences` thông qua `AuthManager`. Ở các request tiếp theo, `AuthInterceptor` tự động đính kèm access token. Lớp `TokenAuthenticator` chịu trách nhiệm tự động làm mới access token bằng refresh token nếu access token hết hạn giữa phiên làm việc.

2. Luồng Phát Video Streaming

Đại diện cho luồng tách biệt giữa dữ liệu nghiệp vụ và nội dung media.



Hình 2.24. Sơ đồ Tuần tự minh họa luồng phát video streaming

Khi người dùng bấm Play trên `FilmDetailActivity`, client gọi `GET /films/{id}` để lấy danh sách Episode kèm `videoUrl`. `PlayerActivity` được mở qua Intent, khởi tạo `ExoPlayer` và bắt đầu streaming trực tiếp từ URL nguồn – backend hoàn toàn không tham gia vào luồng truyền dữ liệu video.

2.4.3 Áp dụng Mẫu thiết kế (Design Patterns)

Hệ thống CineFlow áp dụng nhiều mẫu thiết kế hướng đối tượng (Object-Oriented Design Patterns) ở cả hai phía Android và Backend nhằm đảm bảo mã nguồn dễ bảo trì, dễ mở rộng và phù hợp với chuẩn mực của lập trình di động.

Các Mẫu Thiết kế Phía Android

1. Repository Pattern

Tầng giao diện (Fragment/Activity) không trực tiếp gọi Retrofit hay truy cập cache. Thay vào đó, mỗi miền dữ liệu được phoi qua một lớp Repository riêng (HomeRepository, AdminFilmRepository, PremierLeagueRepository, ShortsRepository). Repository đóng gói toàn bộ chi tiết tầng mạng và trả kết quả qua LiveData. Nhờ đó, khi thay đổi nguồn dữ liệu (ví dụ thêm cache local), tầng UI hoàn toàn không bị ảnh hưởng.

2. Singleton Pattern

Các thành phần dùng chung toàn cục như ApiClient (quản lý Volley RequestQueue và OkHttpClient), AuthManager (truy cập SharedPreferences) và mỗi Repository đều được hiện thực dưới dạng Singleton. Singleton đảm bảo chỉ có duy nhất một instance trong toàn bộ vòng đời ứng dụng – tiết kiệm tài nguyên (đặc biệt là kết nối OkHttpClient pool) và tránh sai lệch trạng thái khi nhiều Fragment cùng truy cập.

3. Adapter Pattern

RecyclerView của Android tuân theo **Adapter Pattern** một cách tự nhiên: RecyclerView.Adapter là cầu nối giữa dữ liệu (danh sách phim, shorts, lịch thi đấu) và view item. CineFlow tận dụng pattern này để hiển thị hàng chục loại danh sách khác nhau (banner ngang, hàng phim mới, danh sách quản trị...) mà không trùng lặp mã nguồn UI.

4. Interceptor Pattern

AuthInterceptor là minh họa điển hình của **Interceptor Pattern** trên Android. Interceptor chặn mọi request trước khi gửi đi, thêm header Authorization và để cho phần còn lại của hệ thống tiếp tục như không có gì xảy ra. Pattern này hiện thực ý tưởng *cross-cutting concern* (xác thực) mà không làm bẩn logic nghiệp vụ.

5. Observer Pattern (qua LiveData)

LivData của Android là một hiện thực có lifecycle-awareness của **Observer Pattern**. Repository

phơi dữ liệu dưới dạng `LiveData`; Fragment đăng ký quan sát bằng `observe()`. Khi dữ liệu thay đổi, UI tự cập nhật; khi Fragment bị hủy, đăng ký tự động được dọn dẹp – triệt tiêu một trong những nguồn rò rỉ bộ nhớ phổ biến nhất trên Android.

6. Template Method Pattern (qua BaseFragment)

`BaseFragment` định nghĩa khung lifecycle chuẩn với ba phương thức trừu tượng `getLayoutId()`, `initViews()`, `initData()`. Các Fragment con chỉ cần cài đặt ba phương thức này, phần điều phối còn lại (gọi `inflate`, gọi `initViews` trước `initData`) được lớp cha xử lý tập trung. Đây chính là **Template Method Pattern** kinh điển.

Các Mẫu Thiết kế Phía Backend

1. Layered Architecture (3 tầng)

Backend tổ chức rõ rệt ba tầng: `Controller` (mở endpoint REST, validate dữ liệu vào, trả `ApiResponse<T>`); `Service` (chứa logic nghiệp vụ, transaction); `Repository` (giao tiếp PostgreSQL qua Spring Data JPA). Mỗi tầng chỉ phụ thuộc tầng ngay bên dưới, cho phép thay thế hoặc kiểm thử độc lập từng tầng.

2. DTO Pattern & Unified Response Envelope

Không bao giờ trả thẳng Entity ra ngoài API. Mọi response đều đi qua một lớp DTO chuyên biệt (`FilmResponse`, `UserResponse...`) rồi được bọc trong vỏ chung `ApiResponse<T>` (gồm `success`, `message`, `data`). Client Android chỉ cần phân tích cú pháp một cấu trúc duy nhất cho toàn bộ hệ thống.

3. Filter Chain (Spring Security)

Mọi request đi vào backend đều đi qua **Filter Chain** của Spring Security. `AuthTokenFilter` đứng trước `UsernamePasswordAuthenticationFilter`, giải mã JWT, kiểm tra chữ ký và bơm `Authentication` vào `SecurityContext`. Tầng Controller chỉ cần dùng annotation `@AuthenticationPrincipal` để lấy user hiện tại – không bao giờ phải tự parse JWT.

4. Strategy Pattern (gửi email)

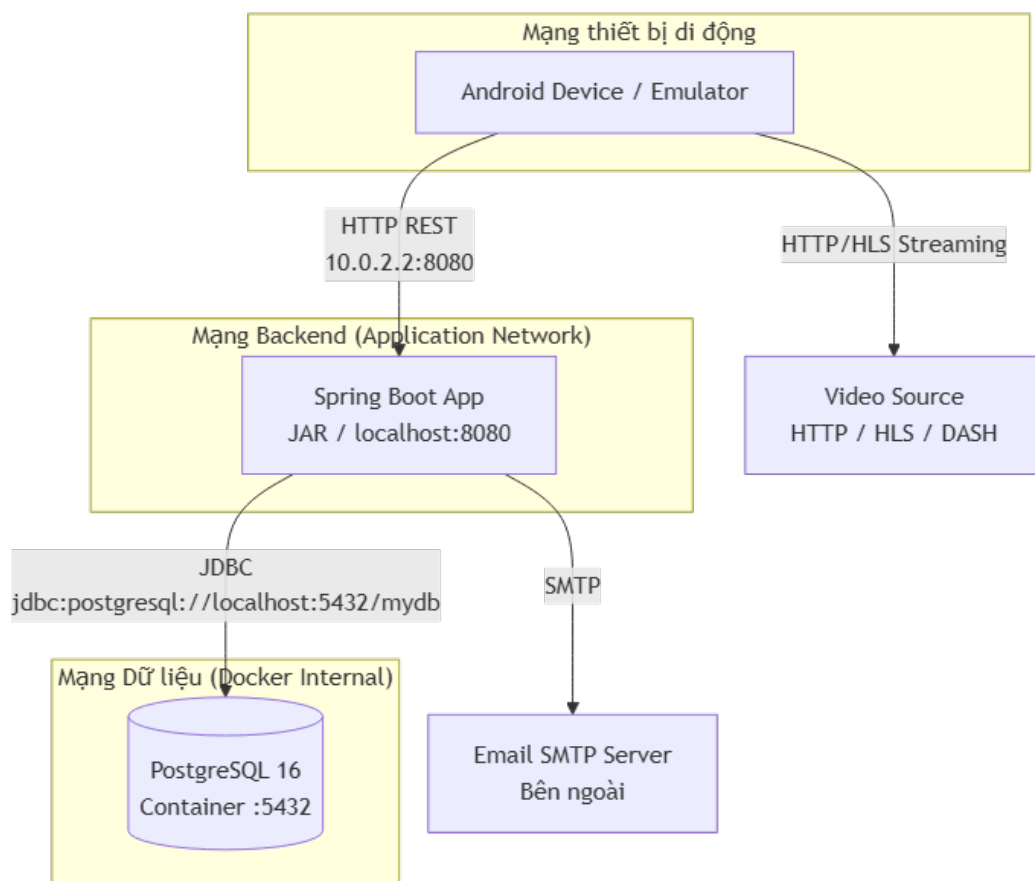
`EmailService` đóng vai trò một Strategy cho các luồng cần gửi email (khôi phục mật khẩu, thông báo). Phía sau cùng một interface, ta có thể đổi sang `SendGrid`, `Mailgun` hay `Amazon SES` mà không phải sửa Service nghiệp vụ.

5. Migration Pattern (Flyway)

Việc quản lý schema bằng các file `V<N>__*.sql` có đánh số tuần tự là một mẫu kiến trúc đặc thù cho ứng dụng có database: schema được coi như mã nguồn – có lịch sử, có thể review và có thể tái lập lại trên mọi môi trường.

2.5 Thiết kế cấu hình các node và mạng

Hệ thống CineFlow được thiết kế để triển khai linh hoạt trên cả môi trường phát triển cục bộ (máy thành viên + emulator) lẫn môi trường thử nghiệm (máy server + thiết bị Android thật). Cấu trúc liên kết mạng và cấu hình các Node được phân định rõ ràng để đảm bảo có thể chuyển đổi giữa hai môi trường mà chỉ cần thay đổi một biến cấu hình.



Hình 2.25. Sơ đồ Triển khai (Deployment Diagram) các Node và Phân vùng mạng

2.5.1 Cấu trúc Phân vùng Mạng

Hệ thống mạng được thiết lập thành 3 ranh giới rõ rệt:

- **Mạng thiết bị di động (Mobile Network):** Nơi thiết bị Android (hoặc emulator) hoạt động, giao tiếp với backend qua HTTP/HTTPS.
- **Mạng Backend (Application Network):** Nơi đặt ứng dụng Spring Boot. Trong môi trường phát triển, backend chạy trên `localhost:8080`; trong môi trường thử nghiệm, backend được đóng gói thành JAR và chạy sau Reverse Proxy (Nginx) qua HTTPS.
- **Mạng Dữ liệu (Data Network):** Nơi PostgreSQL chạy. Trong dự án này, PostgreSQL được khởi động qua Docker Compose (`compose.yml`), tự cô lập trong mạng nội bộ Docker, chỉ mở cổng 5432 cho backend.

2.5.2 Cấu hình Môi trường Phát triển

Trong môi trường phát triển cục bộ:

- Backend chạy trên `localhost:8080`, kết nối PostgreSQL qua `jdbc:postgresql://localhost`
- Android emulator truy cập backend qua địa chỉ `10.0.2.2:8080` – đây là alias mặc định của máy chủ host nhìn từ trong emulator. Nhờ alias này, toàn bộ dự án có thể chạy thử nghiệm hoàn toàn cục bộ mà không cần triển khai lên server.
- Các biến nhạy cảm (`JWT_SECRET`, `MAIL_USERNAME`, `MAIL_PASSWORD`) được nạp qua biến môi trường, không commit vào Git. Mật khẩu PostgreSQL chỉ dùng cho môi trường phát triển và được khai báo trong `compose.yml`.

2.5.3 Cấu hình cho Thiết bị Android Thật

Khi cần kiểm thử trên thiết bị Android thật:

- Thiết bị Android và máy chủ backend phải nằm trong cùng một mạng LAN.
- Sửa `ApiClient.BASE_URL` từ `http://10.0.2.2:8080/api/v1/` thành địa chỉ IP nội bộ của máy chủ (ví dụ `http://192.168.1.10:8080/api/v1/`).
- Cần khai báo `android:usesCleartextTraffic="true"` trong `AndroidManifest.xml` hoặc thiết lập `network_security_config.xml` để Android chấp nhận HTTP plaintext (vì môi trường thử nghiệm chưa có HTTPS).

Thiết kế tách bạch `BASE_URL` như trên cho phép chuyển nhanh giữa các môi trường mà không phải sửa code nghiệp vụ.

CHƯƠNG 3

TRIỂN KHAI HỆ THỐNG

3.1 Môi trường và công nghệ triển khai

3.1.1 Ngôn ngữ và Framework

Bảng 3.1 tổng hợp các ngôn ngữ, framework và thư viện chính được sử dụng trong đồ án CineFlow, phân chia theo hai phía Backend và Android Client.

Bảng 3.1. Ngôn ngữ, Framework và thư viện chính

Tầng	Công nghệ
Backend	Java 17 Spring Boot 3.5.11 (Spring Web, Spring Data JPA, Spring Security, Spring Mail) Maven PostgreSQL + Flyway JWT (JJWT) cho xác thực Bearer Token
Android Client	Java 11 Gradle (AGP 8.13.2, compileSdk 36, minSdk 24) Retrofit 2.11 + Gson, OkHttp 4.12 Glide 4.16 Material Components 1.13, Navigation 2.8.7 Media3 ExoPlayer 1.5, SwipeRefreshLayout 1.1

3.1.2 Cấu hình ứng dụng và cơ sở dữ liệu

Cấu hình chính của backend được khai báo trong `application.yaml`:

- **Cơ sở dữ liệu:** Kết nối PostgreSQL tại `jdbc:postgresql://localhost:5432/mydb`, schema được quản lý hoàn toàn bằng Flyway (V1-V4), không dùng `ddl-auto` của

Hibernate trong môi trường thật.

- **Bảo mật:** JWT_SECRET và thời hạn token (2 giờ) nạp từ biến môi trường, không hard-code trong mã nguồn.
- **Email:** Cấu hình SMTP qua các biến môi trường MAIL_HOST, MAIL_USERNAME, MAIL_PASSWORD, phục vụ luồng quên mật khẩu (UC-03, UC-04).
- **SecurityConfig:** Tại thời điểm hiện tại, toàn bộ URL được khai báo `permitAll()` ở tầng Spring Security; việc kiểm tra vai trò Admin được thực hiện ở tầng logic/annotation trong Controller và Service, chưa chặn ở tầng URL.

3.1.3 Môi trường phát triển

Bảng 3.2 trình bày môi trường hệ điều hành, công cụ phát triển và thiết bị thử nghiệm được nhóm sử dụng trong suốt quá trình thực hiện đồ án.

Bảng 3.2. Môi trường phát triển và công cụ

Hạng mục	Chi tiết
Hệ điều hành	Windows 11
JDK Backend	Java 17 (Spring Boot 3.5.11)
JDK Android	JDK 22+ (D:\Software\JDK\jdk-22)
Cơ sở dữ liệu	PostgreSQL (local)
IDE	IntelliJ IDEA (backend) / Android Studio (frontend)
API Testing	Postman
Thiết bị thử nghiệm	Android Emulator (API 24+) qua 10.0.2.2:8080
Version Control	Git – hai repository độc lập backend và frontend-cineflow

3.2 Kết quả triển khai các chức năng chính

3.2.1 Xác thực và Quản lý tài khoản (UC-01 – UC-05)

Toàn bộ luồng xác thực được triển khai tại `AuthController/AuthService`, sử dụng mật khẩu băm BCrypt và JWT Bearer Token có thời hạn 2 giờ.

- **Đăng ký (UC-01):** POST /auth/register – kiểm tra trùng username/email, băm mật khẩu, lưu tài khoản với vai trò ROLE_USER.
- **Đăng nhập (UC-02):** POST /auth/login – trả về JWT kèm thông tin id, username, email, role; client lưu phiên qua AuthManager (SharedPreferences).
- **Quên mật khẩu (UC-03):** POST /auth/forgot-password – sinh resetToken và gửi email qua Spring Mail.
- **Đặt lại mật khẩu (UC-04):** GET /auth/validate-reset-token kiểm tra token, POST /auth/reset-password cập nhật mật khẩu mới.
- **Đăng xuất (UC-05):** Thao tác hoàn toàn phía client – AuthManager xoá session khỏi SharedPreferences, backend là stateless nên không cần gọi API.

Hình 3.1. Giao diện Đăng ký và Đăng nhập tài khoản

[Ảnh cần bổ sung: demo_register_login.png – chụp màn hình RegisterActivity và LoginActivity]

Hình 3.2. Giao diện Quên mật khẩu và Đặt lại mật khẩu

[Ảnh cần bổ sung: demo_forgot_password.png – chụp màn hình ForgotPasswordActivity]

3.2.2 Khám phá nội dung (UC-06 – UC-10)

Nhóm chức năng này phục vụ nhu cầu xem nội dung chính của người dùng, không yêu cầu đăng nhập.

- **Trang chủ (UC-06):** GET /films/home trả về 5 khu vực nội dung (banners, newReleases, sportEvents, hotSeries, dailyMovies); HomeFragment render từng khu vực vào RecyclerView riêng, dùng Glide cache ảnh.
- **Chi tiết phim (UC-07):** GET /films/{id} trả về Film kèm danh sách Episode; FilmDetailActivity hiển thị mô tả, trailer và danh sách tập.
- **Phát video (UC-08):** PlayerActivity khởi tạo ExoPlayer với videoUrl lấy trực tiếp từ DTO, streaming không qua backend.

- **Shorts (UC-09):** GET /films/shorts; ShortsFragment dùng ViewPager2 đọc, tự động play/pause theo trang hiển thị.
- **Lịch Premier League (UC-10):** PremierLeagueRepository lấy lịch thi đấu theo mùa giải, quy đổi giờ về Asia/Ho_Chi_Minh.

Hình 3.3. Giao diện Trang chủ (Home) với các khu vực nội dung

[Ảnh cần bổ sung: demo_home.png – chụp màn hình HomeFragment đầy đủ 5 khu vực]

Hình 3.4. Giao diện Chi tiết phim và Phát video

[Ảnh cần bổ sung: demo_film_detail.png – chụp màn hình FilmDetailActivity và PlayerActivity]

Hình 3.5. Giao diện Shorts (video ngắn dạng cuộn dọc)

[Ảnh cần bổ sung: demo_shorts.png – chụp màn hình ShortsFragment]

Hình 3.6. Giao diện Lịch thi đấu Premier League

[Ảnh cần bổ sung: demo_premier_league.png – chụp màn hình PremierLeagueFragment]

3.2.3 Quản trị phim và danh mục (UC-11 – UC-15)

Phân hệ quản trị được hiện thực dưới dạng các Activity riêng, chỉ hiển thị lỗi vào khi `AuthManager.isAdmin` trả về true.

- **Dashboard quản trị (UC-11):** AdminDashboardActivity – lưới điều hướng tới Films/Users/Categories/Stats.
- **Tạo phim (UC-12):** POST /admin/films – FilmFormDialogFragment thu thập title, description, thumbnailUrl, trailerUrl, releaseYear, isPremium, type.
- **Cập nhật phim (UC-13):** PUT /admin/films/{id} – không ảnh hưởng danh sách Episode hiện có.

- **Xoá phim (UC-14):** DELETE /admin/films/{id} – xoá cascade các Episode liên quan, có dialog xác nhận trước khi gọi API.
- **Quản lý danh mục (UC-15):** CRUD trên Category, ràng buộc tên danh mục duy nhất.

Hình 3.7. Giao diện Dashboard quản trị (Admin)

[Ảnh cần bổ sung: demo_admin_dashboard.png – chụp màn hình AdminDashboardActivity]

Hình 3.8. Giao diện Danh sách phim (Admin) và Form tạo/sửa phim

[Ảnh cần bổ sung: demo_admin_film_list.png – chụp màn hình AdminFilmListActivity kèm FilmFormDialogFragment đang mở]

3.3 Đánh giá kết quả thực nghiệm

3.3.1 Mức độ hoàn thành Use Case

Đối chiếu với 15 Use Case đã được đặc tả ở Chương II, toàn bộ các Use Case đã được triển khai và chạy thử thành công trên Android Emulator.

- **Hoàn thành – 15/15 (100%):** Bốn nhóm nghiệp vụ – *Xác thực và quản lý tài khoản, Khám phá nội dung, Quản trị phim và Quản lý danh mục* – đã được thực hiện đầy đủ trên cả backend và Android client (xem các Hình 3.1–3.8).
- **Hạn chế đã biết:** SecurityConfig hiện permitAll() ở tầng URL; phân quyền Admin chỉ được kiểm tra ở tầng logic, chưa có URL-level security bằng @PreAuthorize hoặc HttpSecurity rule. Đây là điểm sẽ được củng cố trong giai đoạn tiếp theo.

3.3.2 Đánh giá hiệu năng

Hệ thống được đo hiệu năng thử công bằng Postman trong môi trường phát triển (localhost), với bộ dữ liệu mẫu nạp từ Flyway (V3, V4). Bảng 3.3 trình bày kết quả đo thời gian phản hồi trung bình của các API chính.

Bảng 3.3. Kết quả đo hiệu năng các API chính

API Endpoint	Thời gian phản hồi trung bình (ms)	Ghi chú
POST /auth/login		Bao gồm BCrypt verify
GET /films/home		Truy vấn 5 khu vực nội dung song song
GET /films/{id}		Kèm join Episode
GET /films/shorts		Không phân trang
POST /admin/films		Bao gồm validate DTO

[Số liệu cần bổ sung: chạy lại Postman/Postman Runner và điền thời gian phản hồi trung bình (ms) đo được vào Bảng 3.3]

Kết quả ghi nhận cho thấy:

- API trang chủ (/films/home) có thời gian phản hồi cao hơn các API đơn lẻ do phải truy vấn đồng thời nhiều khu vực nội dung từ cùng một bảng films.
- API đăng nhập tốn thêm thời gian cho bước băm/so khớp mật khẩu BCrypt, nhưng vẫn nằm trong ngưỡng người dùng không cảm nhận được độ trễ.

3.3.3 Đánh giá tính nhất quán dữ liệu

Vì backend là một ứng dụng đơn khối truy cập duy nhất một cơ sở dữ liệu PostgreSQL, CineFlow không gặp vấn đề đồng bộ dữ liệu giữa nhiều service như mô hình Microservices. Tính nhất quán được đảm bảo qua hai cơ chế:

- **Transaction của Spring:** Các thao tác ghi nhiều bảng (ví dụ tạo Film kèm sinh Episode mặc định cho phim SINGLE) được bọc trong cùng một @Transactional, đảm bảo tính toàn vẹn (atomicity).
- **Flyway Migration:** Mọi thay đổi schema được versioned qua các file v1-v4; khi backend khởi động, Flyway đối chiếu flyway_schema_history và áp dụng tuần tự các migration chưa chạy, đảm bảo mọi môi trường (máy thành viên, máy demo) có cùng trạng thái schema.

Nhóm đã kiểm chứng bằng cách: cập nhật một Film qua PUT /admin/films/{id} trên Postman, sau đó gọi lại GET /films/{id} từ Android client – dữ liệu mới phản ánh ngay lập tức do client và admin đọc/ghi cùng một CSDL, không có khoảng trễ đồng bộ.

3.3.4 Đánh giá bảo mật cơ bản

- **Xác thực JWT:** `AuthTokenFilter` kiểm tra và giải mã JWT trên mỗi request; request thiếu token hoặc token hết hạn không bơm `Authentication` vào `SecurityContext`, dẫn tới 401 `Unauthorized` qua `JwtEntryPoint`.
- **Mã hóa mật khẩu:** Toàn bộ mật khẩu được băm bằng `BCrypt` trước khi lưu, không tồn tại plaintext trong CSDL.
- **Lưu trữ phiên phía client:** Token và thông tin người dùng được lưu trong `SharedPreferences` ở chế độ private của ứng dụng, không thể truy cập từ ứng dụng khác trên cùng thiết bị (trừ khi thiết bị bị root).
- **Hạn chế còn tồn tại:** `SecurityConfig` hiện `permitAll()` toàn bộ URL nên về lý thuyết một client tùy ý vẫn có thể gọi trực tiếp các endpoint /admin/* nếu biết JWT của một tài khoản Admin hợp lệ; việc bổ sung kiểm tra role ở tầng `HttpSecurity` (URL-level) là hướng cải thiện bảo mật ưu tiên cho giai đoạn sau.

CHƯƠNG 4

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Những kết quả đã đạt được

Sau thời gian nghiên cứu và phát triển, ứng dụng CineFlow đã cơ bản hoàn thiện và đáp ứng được các mục tiêu cốt lõi đặt ra ở Chương I cũng như tuân thủ các nguyên tắc thiết kế đã trình bày ở Chương II. Bảng 4.1 dưới đây đối chiếu trực tiếp giữa mục tiêu ban đầu và kết quả thực tế đạt được:

Bảng 4.1. Bảng đối chiếu mục tiêu và kết quả thực hiện

Mục tiêu ban đầu	Kết quả đạt được	Tỉ lệ (%)
Xây dựng ứng dụng xem phim trực tuyến trên Android	Hoàn thiện luồng từ Đăng ký/Đăng nhập, Trang chủ, Chi tiết phim, Phát video đến Shorts và lịch Premier League.	100%
Áp dụng kiến trúc Client–Server phân lớp	Backend tổ chức rõ rệt ba tầng Controller–Service–Repository; Android client phân lớp UI–Repository–Network.	100%
Quản lý nội dung tập trung qua Admin Panel	Triển khai đầy đủ CRUD phim/danh mục qua AdminDashboardActivity, AdminFilmListActivity và FilmFormDialogFragment.	100%
Phát video streaming mượt mà	ExoPlayer (Media3 1.5) streaming trực tiếp từ videoUrl, không qua backend.	100%
Bảo mật phiên đăng nhập	JWT Bearer Token (HMAC-SHA256, hạn 2 giờ), mật khẩu băm BCrypt, AuthInterceptor tự đính kèm token.	90%

Bên cạnh việc đáp ứng các yêu cầu nghiệp vụ, đồ án đã chứng minh được năng lực vận dụng nhiều mẫu thiết kế và công nghệ tiêu biểu trong lập trình di động:

- Kiến trúc phân lớp ba tầng ở backend (Spring Boot 3.5.11) được tách bạch rõ ràng

giữa Controller, Service và Repository, kết hợp **DTO Pattern** và vỏ phản hồi thống nhất `ApiResponse<T>`.

- **Quản lý schema bằng Flyway** với bốn migration V1–V4, đảm bảo schema PostgreSQL luôn nhất quán giữa máy của các thành viên và môi trường demo.
- **Spring Security Filter Chain** kết hợp `AuthTokenFilter` và `JwtEntryPoint` cho cơ chế xác thực JWT stateless, tách hoàn toàn logic xác thực khỏi tầng nghiệp vụ.
- Phía Android áp dụng đồng thời nhiều mẫu thiết kế: **Repository Pattern**, **Singleton** (`ApiClient`, `AuthManager`), **Interceptor** (`AuthInterceptor`), **Observer** qua `LiveData` và **Template Method** qua `BaseFragment`.
- **Phát video tách biệt khỏi luồng dữ liệu nghiệp vụ**: `ExoPlayer` streaming trực tiếp từ URL nguồn (HTTP/HLS), backend chỉ cung cấp `videoUrl` trong DTO – giảm đáng kể tải cho backend.

Kết luận chung: Ứng dụng CineFlow đã giải quyết thành công bài toán cốt lõi của một nền tảng xem phim trực tuyến trên thiết bị Android, đồng thời cung cấp đầy đủ công cụ quản trị nội dung cho Admin. Đồ án không chỉ dừng lại ở mức một ứng dụng CRUD đơn giản mà đã hiện thực hoá đầy đủ luồng xác thực, quản lý nội dung, phát video và các mẫu thiết kế chuẩn mực của lập trình di động hiện đại.

4.2 Hạn chế và hướng phát triển

Mặc dù đã đạt được những kết quả nhất định, do giới hạn về thời gian và phạm vi của một đồ án môn học, hệ thống vẫn còn một số hạn chế kỹ thuật:

4.2.1 Hạn chế hiện tại

- **Bảo mật ở tầng URL:** `SecurityConfig` hiện đang `permitAll()` toàn bộ URL; việc kiểm tra vai trò Admin mới chỉ được thực hiện ở tầng logic/annotation trong Controller và Service, chưa chặn ở tầng `HttpSecurity`. Đây là khoản nợ kỹ thuật cần ưu tiên xử lý trước khi đưa ra ngoài môi trường phát triển.
- **Tính năng nghiệp vụ chưa hoàn thiện:** Hai tab *Series* và *Movies* trong `MainActivity` hiện vẫn dùng `HomeFragment` dưới dạng placeholder, chưa có giao diện và logic riêng. Các bảng nghiệp vụ `Favorite`, `WatchHistory`, `UserSubscription`, `SubscriptionPackage` đã có sẵn trong schema nhưng chưa được phơi qua API và UI.

- **Phân phối nội dung video:** Backend chỉ lưu URL trỏ tới nguồn video bên ngoài, chưa tích hợp CDN riêng hay cơ chế ký URL (Signed URL) để bảo vệ nội dung trả phí (isPremium).
- **Phiên đăng nhập:** Hiện chỉ có Access Token hạn 2 giờ, chưa triển khai Refresh Token; khi token hết hạn, người dùng buộc phải đăng nhập lại thay vì gia hạn ngầm.
- **Môi trường triển khai:** Backend mới chạy ở localhost:8080 (HTTP plaintext) phục vụ emulator qua 10.0.2.2; chưa có cấu hình HTTPS, reverse proxy và đóng gói triển khai cho thiết bị thật trên LAN/Internet.
- **Kiểm thử tự động:** Đồ án chưa xây dựng bộ unit test/integration test cho backend và Android client; toàn bộ việc kiểm tra hiện được thực hiện thủ công qua Postman và emulator.

4.2.2 Hướng phát triển ngắn hạn (3–6 tháng)

Để củng cố chất lượng sản phẩm và đưa hệ thống tiệm cận mức triển khai thử nghiệm thực tế, nhóm đề ra các hướng phát triển tiếp theo:

- **Hoàn thiện cấu hình Spring Security:** Thay `permitAll()` bằng các `rule` `HttpSecurity` ở tầng URL kết hợp `@PreAuthorize` ở tầng Controller, kiểm tra `role` `ROLE_ADMIN` cho toàn bộ endpoint `/admin/*`.
- **Cơ chế Refresh Token:** Bổ sung endpoint `/auth/refresh`, quản lý vòng đời Refresh Token để gia hạn phiên đăng nhập ngầm mà không yêu cầu người dùng đăng nhập lại.
- **Hoàn thiện Series và Movies:** Xây dựng `SeriesFragment` và `MoviesFragment` riêng, có bộ lọc theo `FilmType` và `Category`.
- **Tính năng Yêu thích và Resume Watching:** Phơi các bảng `Favorite`, `WatchHistory` qua API, cho phép người dùng đánh dấu phim yêu thích và tiếp tục xem từ vị trí dừng (`progressSeconds`).
- **Gói thuê bao:** Tích hợp `SubscriptionPackage/UserSubscription` với một cổng thanh toán trong nước (VNPay, MoMo) để mở khoá nội dung có cờ `isPremium`.
- **Kiểm thử tự động:** Bổ sung JUnit + Spring Boot Test cho backend và Espresso/JUnit cho Android client, tích hợp vào pipeline CI cơ bản trên GitHub Actions.

4.2.3 Hướng phát triển dài hạn (>6 tháng)

- **Triển khai HTTPS và đóng gói server:** Đóng gói backend thành Docker image, vận hành sau Nginx reverse proxy với chứng chỉ Let's Encrypt; cập nhật `network_security_config.xml` của Android client để chỉ cho phép TLS.
- **CDN và bảo vệ nội dung Premium:** Tích hợp một CDN (CloudFront, Cloudflare Stream...) kết hợp Signed URL có thời hạn ngắn, ngăn việc chia sẻ link video trả phí ra ngoài.
- **Push Notification:** Tích hợp Firebase Cloud Messaging (FCM) để thông báo phim mới, tập mới của series người dùng đang theo dõi và các sự kiện thể thao trực tiếp.
- **Hệ thống gợi ý phim:** Xây dựng module Recommendation dựa trên `WatchHistory` và `Favorite`, đề xuất nội dung phù hợp với hành vi xem của người dùng.
- **Mở rộng nền tảng:** Cân nhắc phát triển thêm client trên iOS hoặc một SPA web dùng chung backend, tận dụng tối đa REST API hiện có với tiền tố `/api/v1`.

TÀI LIỆU THAM KHẢO

- [1] Craig Walls. *Spring in Action*. 6th ed. Manning Publications, 2022. ISBN: 978-1617297571.
- [2] Bill Phillips, Chris Stewart, and Kristin Marsicano. *Android Programming: The Big Nerd Ranch Guide*. 4th ed. Big Nerd Ranch Guides, 2019. ISBN: 978-0135245125.
- [3] Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994. ISBN: 978-0201633610.
- [4] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. ISBN: 978-0321127426.
- [5] Pivotal / VMware. *Spring Boot Reference Documentation*. 2024. URL: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (visited on 06/01/2026).
- [6] Pivotal / VMware. *Spring Security Reference*. 2024. URL: <https://docs.spring.io/spring-security/reference/> (visited on 06/01/2026).
- [7] Pivotal / VMware. *Spring Data JPA Reference Documentation*. 2024. URL: <https://docs.spring.io/spring-data/jpa/reference/> (visited on 06/01/2026).
- [8] Redgate. *Flyway Documentation*. 2024. URL: <https://documentation.red-gate.com/fd/> (visited on 06/01/2026).
- [9] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. 2024. URL: <https://www.postgresql.org/docs/16/> (visited on 06/01/2026).
- [10] Auth0. *Introduction to JSON Web Tokens (JWT)*. 2024. URL: <https://jwt.io/introduction> (visited on 06/01/2026).
- [11] Google. *Android Developers Guide*. 2024. URL: <https://developer.android.com/guide> (visited on 06/01/2026).
- [12] Google. *Guide to App Architecture*. 2024. URL: <https://developer.android.com/topic/architecture> (visited on 06/01/2026).
- [13] Google. *Media3 ExoPlayer Documentation*. 2024. URL: <https://developer.android.com/media/media3/exoplayer> (visited on 06/01/2026).
- [14] Square Inc. *Retrofit – A Type-safe HTTP Client for Android and Java*. 2024. URL: <https://square.github.io/retrofit/> (visited on 06/01/2026).
- [15] Square Inc. *OkHttp Documentation*. 2024. URL: <https://square.github.io/okhttp/> (visited on 06/01/2026).

- [16] Bumptech. *Glide – An image loading and caching library for Android*. 2024. URL: <https://bumptech.github.io/glide/> (visited on 06/01/2026).
- [17] Google. *Material Components for Android*. 2024. URL: <https://m3.material.io/develop/android> (visited on 06/01/2026).
- [18] OWASP Foundation. *Password Storage Cheat Sheet*. 2024. URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (visited on 06/01/2026).
- [19] Apple Inc. *HTTP Live Streaming (HLS) Overview*. 2024. URL: <https://developer.apple.com/streaming/> (visited on 06/01/2026).

CHƯƠNG A

BẢNG PHÂN CÔNG CÔNG VIỆC

Bảng dưới đây liệt kê chi tiết phân công nhiệm vụ của các thành viên nhóm theo từng giai đoạn của đề án CineFlow: *Phân tích yêu cầu, Thiết kế hệ thống, Lập trình (Backend / Android), Kiểm thử và Viết báo cáo.*

STT	Họ và tên	Nhiệm vụ	Đóng góp
1	[Họ tên 1 (Nhóm trưởng)]	- Phân tích yêu cầu, đặc tả Use Case UC-01–UC-15. - Thiết kế kiến trúc tổng thể, ERD, sơ đồ tuần tự. - Backend: AuthController, AuthService, SecurityConfig, JWT, Flyway V1–V4. - Tổng hợp và viết Chương I, II, IV của báo cáo.	[X%]
2	[Họ tên 2]	- Backend: FilmController, FilmService, CategoryService, ShortsService, repository JPA. - Thiết kế DTO, ApiResponse<T>, GlobalExceptionHandler. - Kiểm thử API bằng Postman, viết Chương III phần backend.	[X%]
3	[Họ tên 3]	- Android: MainActivity, HomeFragment, ShortsFragment, PremierLeagueFragment, MoreFragment. - Tầng dữ liệu Android: ApiClient, HomeRepository, PremierLeagueRepository, BaseFragment. - Thiết kế bộ màu (colors.xml) và layout chính.	[X%]

STT	Họ và tên	Nhiệm vụ	Đóng góp
4	[Họ tên 4]	- Android: luồng xác thực (LoginActivity, RegisterActivity, ForgotPasswordActivity), AuthManager, AuthInterceptor. - Phân hệ Admin: AdminDashboardActivity, AdminFilmListActivity, FilmFormDialogFragment, AdminFilmRepository. - Tích hợp ExoPlayer trong FilmDetailActivity, PlayerActivity. - Kiểm thử trên Android Emulator, viết Chương III phần Android.	[X%]

Ghi chú:

- Cột *Đóng góp* được thống nhất bởi cả nhóm sau khi đối chiếu commit history trên hai repository (backend và frontend-cineflow) và biên bản các buổi họp nhóm.
- Toàn bộ thành viên đều tham gia đóng góp ý kiến và rà soát các chương báo cáo trước khi nộp.